Computer Methods and Programs in Biomedicine 276 (2026) 109187

Contents lists available at ScienceDirect

Computer Methods and Programs in Biomedicine

journal homepage: https://www.sciencedirect.com/journal/computer-methods-and-programs-in-biomedicine

Random forests for individual treatment effect estimation with the R package ITERF

Sami Tabib, Denis Larocque *

Department of Decision Sciences, HEC Montréal, 3000 chemin de la Côte–Sainte–Catherine, Montréal (Québec), Canada, H3T 2A7

ARTICLE INFO

Keywords:
Individual treatment effect
Conditional average treatment effect
Heterogeneous treatment
Continuous treatment
Maximum treatment effect
Survival data
Random forest
Tree-based method
R package

ABSTRACT

Background and Objectives: Treatment effects often vary across individuals within a population. In contexts such as personalized medicine, it is crucial to accurately estimate treatment effects at the individual level. Random forests are among the most popular, versatile, and efficient statistical learning methods. This article introduces the R package ITERF, designed to estimate individual treatment effects using random forests across various settings. In particular, new methods to estimate the maximum treatment effect are introduced.
Methods and Results: The ITERF package provides methods for estimating treatment effects in two scenarios: (1) survival outcomes with right-censoring and a binary treatment, and (2) continuous outcomes with a continuous treatment. All methods are based on random forests. A simulation study demonstrates that the proposed methods for estimating the maximum treatment effect perform as expected and show considerable promise. An illustration, using real data, that explores the link between sleep duration and cognitive health in the elderly is given.
Conclusion: The ITERF package offers a fast and user-friendly tool for estimating treatment effect measures using random forests, making it a valuable resource for researchers and practitioners in personalized treatment evaluation.

1. Introduction

A common objective in many studies is to investigate the effect of a treatment (G) on an outcome of interest (Y) within a population. The term "treatment" is used broadly here to encompass various scenarios. For instance, it may refer to a medical intervention, a marketing campaign, exposure to a pollutant, or a behavior such as smoking. In the first two cases, data are often obtained from randomized trials, whereas in the latter two, studies typically rely on observational data. Treatment effects are frequently heterogeneous across the population and may vary depending on individual characteristics (covariates) denoted by $X = (X_1, \ldots, X_p)$.

For simplicity, we assume that Y is either continuous (or treated as such) or binary. For a fixed value of $X = x$, the general treatment function is defined as

$$h_x(g) = E[Y|G = g, X = x].$$

While estimating the full treatment function for each value of x may be of interest, the focus is typically on specific scalar summaries of h_x. One commonly studied quantity is the Conditional Average Treatment Effect (CATE). In the case of a binary treatment, where $G = 1$ denotes treated individuals and $G = 0$ denotes controls, the CATE is defined as:

$$\tau_0(x) = E[Y|G = 1, X = x] - E[Y|G = 0, X = x] = h_x(1) - h_x(0). \quad (1)$$

This quantity is also referred to as Heterogeneous Treatment Effect (HTE) or Individual Treatment Effect (ITE). A recent review on this topic is provided by [1]. In marketing, this type of problem is commonly known as "Uplift Modeling" [2]. To give a meaningful interpretation to definition (1), a theoretical framework is required. The most widely used is the potential outcomes framework [3].

From a statistical perspective, heterogeneity in treatment effects implies the presence of interactions between the treatment and covariates in modeling the response. Consequently, the CATE can be estimated using traditional parametric models by including interaction terms. However, specifying the correct interactions is challenging, particularly in high-dimensional settings where the number of covariates is large. In such cases, methods capable of automatically capturing complex interactions are especially useful. Statistical and machine learning approaches are well suited for this task. This has led to growing interest in developing methods for CATE estimation [4,5].

* Corresponding author.
E-mail addresses: sami.tabib@hec.ca (S. Tabib), denis.larocque@hec.ca (D. Larocque).

https://doi.org/10.1016/j.cmpb.2025.109187
Received 23 July 2025; Received in revised form 19 November 2025; Accepted 29 November 2025
Available online 16 December 2025

Random forests [6] are among the most widely used statistical learning methods, particularly appreciated by practitioners for their strong predictive performance, ease of use, and broad availability. Like many other learning algorithms, random forests can be applied directly to estimate the CATE within general frameworks such as T-learning and X-learning [7]. However, specialized adaptations of the tree-building algorithm have been developed specifically for CATE estimation. A prominent example is the causal random forest method [8,9], implemented in the R package `grf` [10]. A review of the use of causal forests in peer-reviewed literature is provided in [11], highlighting their widespread adoption. In causal forests, trees are constructed using a custom splitting rule designed to maximize heterogeneity in the estimated treatment effects. Other specialized methods have also been proposed. For instance, [12] developed a method tailored to survival outcomes with right-censoring under binary treatment, while [13] explored random forest approaches for CATE estimation with continuous treatments. However, the methods introduced in these two articles were not, at the time of publication, available in a publicly released R package.

The goal of this article is to introduce a new R package, `ITERF` that will include the methods developed in [12,13]. The package provides a convenient, user-friendly, and fast (with parallel computing support) of these methods that were not publicly available before. In addition, we introduce two new random forest methods to estimate the maximum effect with a continuous treatment, which are also included in the package. These are the first random forest methods proposed for this specific problem. As such, they are investigated in details with simulation studies and real data examples. Appendix A contains more details about the required assumptions for these methods.

2. R package `ITERF`

This section presents the features of the R package `ITERF` and the methods it implements.

2.1. Introduction and availability

The R package `randomForestSRC` [14–16] is a well-established and widely used implementation of random forests. It is actively maintained, offers a rich set of features, and continues to evolve with regular updates. The package supports various problem types, including regression, classification, and survival analysis. One particularly convenient feature of `randomForestSRC` is its support for custom split rules. This allows users to define new splitting criteria, code them in C and integrate them into the package for any of the supported problem classes. As a result, users can leverage the package's efficient tree-building and random forest infrastructure without needing to reimplement it from scratch. In developing the `ITERF` package, however, we needed to take a step further by defining a new problem class specifically for CATE estimation, as the existing classes in `randomForestSRC` were not suitable for this purpose.

The `ITERF` package is available on GitHub and can be installed using:

```
devtools::install_github("Samitbmtl/ITERF")
```

The source code is accessible at:

https://github.com/Samitbmtl/ITERF

The following sections provide a detailed overview of the features and methods implemented in `ITERF`. Table 1 offers a summary for quick reference. Code examples illustrating the use of the package are provided in Appendix B and on the GitHub page. In addition, vignettes will be added in the future. The simulation study and the real data example are performed with R version 4.5.1 and `ITERF` version 1.0.0.

2.2. General features of the package

In all cases, we consider a response variable Y, a treatment variable G, and a set of covariates $X = (X_1, \ldots, X_p)$. In the specific case of survival data, an additional censoring indicator C is included, where $C = 1$ denotes an observed event and $C = 0$ indicates a right-censored observation.

As mentioned earlier, a new problem class was developed to enable the necessary information to be passed to the split evaluation module. This new class, called "Incremental", is characterized by representing the response using a pair of variables, Y and G, via the syntax `f(Y, G)`. For survival data, the censoring indicator C is also included, resulting in the syntax `f(Y, C, G)`.

The `ITERF` package retains the core tree-growing controls and tuning of `randomForestSRC`. In particular, the key parameters `nodesize` and `mtry` remain available. The syntax of `ITERF` closely mirrors that of its parent package, facilitating a smooth transition for users.

The modeling workflow follows the standard approach: a random forest is trained on a dataset, and predictions are obtained using a dedicated prediction function. However, the prediction mechanism in `ITERF` differs from that of the source package, as it employs the Bag of Observations for Prediction (BOP) strategy described in Section 2.7.

The core tree-building process is implemented in C, using dynamic memory allocation to construct the tree and forest objects. Once the forest is built, it is returned to R. When the prediction function is called, the R code verifies that the covariates match and then invokes the C code to compute predictions using the BOP method.

`ITERF` also supports parallel processing to enhance efficiency during training and prediction, especially with large datasets. Parallel execution in the C code is implemented using the OpenMP API [17]. Although more complex to implement, this approach significantly improves performance by leveraging multiple CPU cores. By default, the package utilizes all available cores, but users can restrict the number of threads if desired.

2.3. Heterogeneity maximization split rules

The split rule is a central component of any tree-building algorithm. In the `ITERF` package, some split rules are designed to maximize heterogeneity in the estimation of the parameter of interest. At a given node t, let t_L and t_R denote the sets of indices corresponding to observations assigned to the left and right child nodes, respectively, for a candidate split. Let N_L and N_R represent the number of observations in the left and right nodes. Let τ denote the parameter of interest, for example the CATE $\tau_0(x)$ defined in (1). During tree construction, it is assumed that observations within a node form a homogeneous group with respect to τ. Let $\hat{\tau}_L$ and $\hat{\tau}_R$ be estimates of τ computed using only the observations in the left and right nodes, respectively. The split rule used to promote heterogeneity in the parameter estimates is given by:

$$\sqrt{N_R N_L}|\hat{\tau}_L - \hat{\tau}_R|. \tag{2}$$

The optimal split is the one that maximizes this criterion.

2.4. Survival data with a binary treatment

A novel method for estimating the CATE in the context of survival data with right-censoring is proposed in [12]. The treatment variable G is binary, taking the value 1 for treated subjects and 0 for controls. The observed response is defined as $T = \min(Y, V)$, where Y is the event time and V is the censoring time. The parameter of interest is:

$$\tau_1(x) = E[Y|G = 1, X = x] - E[Y|G = 0, X = x]. \tag{3}$$

To estimate τ_1 within a node, the method relies on the fact that the expected value of a non-negative random variable can be expressed as the integral of its survival function S, i.e., $E[Y] = \int_0^\infty S(y)dy$. In this approach, the Kaplan–Meier (KM) estimator is used to estimate

Table 1
Summary of the main methods and functions available in the package ITERF.

Function name	Article	Response	Treatment	Parameter of interest
HETSurv	[12]	Survival with right-censoring	Binary	$\tau_1(x)$ (3)
HET	[13]	Continuous	Continuous	$\tau_2(x)$ (4)
CMB	[13]	Continuous	Continuous	$\tau_2(x)$ (4)
HETMaxQuad	New method	Continuous	Continuous	$\tau_3(x)$ (8)
CMBMaxQuad	New method	Continuous	Continuous	$\tau_3(x)$ (8)

the survival functions. However, since the KM estimator is undefined beyond the largest observed time, an optional exponential tail extension is provided, as described in [12]. We provide the details in Appendix C for completeness.

Let $\hat{S}_L^T(y)$ and $\hat{S}_L^C(y)$ denote the (possibly extended) KM estimators of the survival functions for the treated ($G = 1$) and control ($G = 0$) groups in the left node, respectively. Similarly, let $\hat{S}_R^T(y)$ and $\hat{S}_R^C(y)$ represent the corresponding estimators in the right node. The CATE estimate in the left node is given by

$$\hat{\tau}_{1L} = \int_0^\infty (\hat{S}_L^T(y) - \hat{S}_L^C(y))dy.$$

It represents the estimated mean survival time for a treated subject minus the estimated mean survival time for a control subject in the left node. Similarly, CATE estimate in the right node is given by

$$\hat{\tau}_{1R} = \int_0^\infty (\hat{S}_R^T(y) - \hat{S}_R^C(y))dy.$$

Tree construction is then guided by the heterogeneity-based split rule defined in Eq. (2): $\sqrt{N_R N_L}|\hat{\tau}_{1L} - \hat{\tau}_{1R}|$. This method is referred to as ITE SRF in [12].

2.5. Slope estimation with a continuous response and treatment

CATE estimation for a continuous response and treatment is studied in [13], where the following model is assumed:

$$E[Y|G = g, X = x] = \mu(x) + \tau_2(x)g, \tag{4}$$

and the quantity of interest is $\tau_2(x)$. This formulation assumes that the response is a linear function of the treatment for any x, but with varying coefficients. In this context, $\tau_2(x)$ quantifies the expected change in the response associated with a one-unit increase in the treatment, conditional on covariates x.

The ITERF package provides two methods for estimating $\tau_2(x)$. Consider the generic linear model:

$$Y = \mu + \tau_2 G + \epsilon \tag{5}$$

which relates the response Y to the treatment G. For a candidate split in a tree, model (5) is fitted using only the observations in the left node, yielding estimates $\hat{\mu}_L$ and $\hat{\tau}_{2L}$. The same model is fitted to the observations in the right node, producing $\hat{\mu}_R$ and $\hat{\tau}_{2R}$.

The first method, referred to as HET in [13], selects splits that maximize heterogeneity in the treatment effect. The split rule is: $\sqrt{N_R N_L}|\hat{\tau}_{2L} - \hat{\tau}_{2R}|$, where larger value indicate better splits.

The second split rule focuses on predictive accuracy for Y. The best split is the one minimizing

$$\sum_{i\in t_L}(Y_i - \hat{\mu}_L - \hat{\tau}_{2L}G_i)^2 + \sum_{i\in t_R}(Y_i - \hat{\mu}_R - \hat{\tau}_{2R}G_i)^2. \tag{6}$$

The resulting forest is used to estimate $\tau_2(x)$. This method is referred to as CMB in [13] and is an example of the proxy target approach.

In addition, local centering options are available for the treatment and the response, which can often improve the estimation performance, as demonstrated in the simulation study of [13]. Let $\tilde{Y}_c$ be the centered version of Y obtained as $\tilde{Y}_c = Y - \hat{m}(x)$ where $\hat{m}(x)$ is an estimation of the conditional mean of the response $m(x) = E[Y|X = x]$. Let $\tilde{G}_c$ be the centered version of G obtained as $\tilde{G}_c = G - \hat{\pi}(x)$ where $\hat{\pi}(x)$ is an estimation of the conditional mean of the treatment $\pi(x) = E[G|X = x]$. These estimates are obtained using Out-of-Bag (OOB) predictions from random forests built with the randomForestSRC package. This centering is performed as a preprocessing step, after which the centered variables $\tilde{Y}_c$ and $\tilde{G}_c$ are used in the forest construction for both estimation methods.

2.6. Maximum effect estimation with a continuous response and treatment

As noted in the conclusion of [13], assuming model (4) can serve as a reasonable approximation in many practical situations. However, this model implicitly assumes that the maximum treatment effect occurs at one of the boundaries of the treatment range. Consequently, if the objective is to estimate the maximum effect precisely, a more flexible modeling approach may be preferable. As a simple alternative, [13] proposes the following model:

$$E[Y|G = g, X = x] = \beta_0(x) + \beta_1(x)g + \beta_2(x)g^2. \tag{7}$$

In this article, we assume that the treatment variable is scaled such that $G \in [0, 1]$. Given covariates $X = x$, the quantity of interest is the maximum treatment effect, denoted by $\tau_3(x)$, and defined as follows:

$$\tau_3(x) = \max_{0\le g\le 1} E[Y|G = g, X = x]. \tag{8}$$

Estimating the maximum treatment effect can be valuable, for instance, in identifying individuals for whom the treatment holds the greatest potential benefit. It is important to emphasize, however, that this is distinct from the problem of determining the optimal treatment level that yields the best expected response, formally $\text{argmax}_{0\le g\le 1} E[Y|G = g, X = x]$ for a given x. The latter is more closely related to dose-finding problems in treatment optimization.

Let

$$Y = \beta_0 + \beta_1 G + \beta_2 G^2 + \epsilon \tag{9}$$

be a generic quadratic model relating the response Y to the treatment G. For model (9), the maximum effect can be explicitly written as:

$$\tau_3 = \begin{cases} \beta_0 - \frac{\beta_1^2}{4\beta_2} & \text{if } \beta_2 < 0 \text{ and } 0 \le -\beta_1/(2\beta_2) \le 1 \\ \max\{\beta_0, \beta_0 + \beta_1 + \beta_2\} & \text{otherwise.} \end{cases} \tag{10}$$

For a candidate split in a tree, model (9) is fitted using only the observations in the left node, yielding estimates $\hat{\beta}_{0L}$, $\hat{\beta}_{1L}$ and $\hat{\beta}_{2L}$. Based on these estimates, the estimated maximum effect for the left node, denoted by $\hat{\tau}_{3L}$, is derived from (10). The same procedure is applied to the observations in the right node, resulting in the estimates $\hat{\beta}_{0R}$, $\hat{\beta}_{1R}$, $\hat{\beta}_{2R}$ and $\hat{\tau}_{3R}$.

The first method for estimating the maximum effect, called HETMaxQuad, uses a heterogeneity maximization split rule. The splitting criterion for this method is given by $\sqrt{N_R N_L}|\hat{\tau}_{3L} - \hat{\tau}_{3R}|$, where larger values indicate better splits.

The second method, called CMBMaxQuad, is based on a split rule that focuses on predictive accuracy for Y. The best split is the one minimizing

$$\sum_{i\in t_L}(Y_i - \hat{\beta}_{0L} - \hat{\beta}_{1L}G_i - \hat{\beta}_{2L}G_i^2)^2 + \sum_{i\in t_R}(Y_i - \hat{\beta}_{0R} - \hat{\beta}_{1R}G_i - \hat{\beta}_{2R}G_i^2)^2. \tag{11}$$

The resulting forest is used to estimate $\tau_3(x)$.

2.7. Computation of the final random forest estimate

This section describes how the parameter of interest is estimated for a new observation once a random forest of B trees has been constructed. Specifically, we aim to estimate $\tau_k(x_{new})$, for $k = 1, 2$ or 3, at a new covariate point x_{new}. Let $S_b(x_{new})$ denote the set of training observations that fall into the same terminal node as x_{new} in the bth tree. The `ITERF` package offers three variants for defining $S_b(x_{new})$:

1. All In-Bag Observations: Includes all In-Bag observations, allowing duplicates if an observation appears multiple times due to bootstrap sampling.
2. Unique In-Bag Observations: Includes only one copy of each In-Bag observation per tree, removing duplicates.
3. Out-of-Bag (OOB) Observations: Includes only the OOB observations.

When bootstrap sampling is used to construct trees, it is possible for a training observation to appear multiple times in a terminal node. The distinction between variants 1 and 2 lies in whether these duplicates are retained (variant 1) or removed (variant 2). The union of all such sets across the forest defines the Bag of Observations for Prediction (BOP), formally given by:

$$\text{BOP}(x_{new}) = \bigcup_{b=1}^{B} S_b(x_{new}).$$

This BOP serves as the basis for estimating the parameter of interest. For example, to estimate $\hat{\tau}_2(x_{new})$, model (5) is fitted using the observations $\text{BOP}(x_{new})$, and the resulting estimate $\hat{\tau}_2$ is taken as $\hat{\tau}_2(x_{new})$.

When making predictions for a training data point, the BOP is constructed using only the OOB observations.

2.8. Relationship with the packages `grf` and `randomForestSRC`

The package `randomForestSRC` does not provide direct estimation of individual treatment effect. As explained above, the new package `ITERF` leverages the efficient random forest construction framework of `randomForestSRC` to implement various methods for estimating individual treatment effects. The package `grf` can estimate individual treatment effects in some settings. In the survival data setting of Section 2.4, `ITERF` estimates the quantity $\tau_1(x)$ defined in Eq. (3), representing the difference in mean survival time between treated and control subjects. In contrast, the `grf` package, via its `causal_survival_forest` function, offers two alternative estimands: (i) the difference in restricted mean survival time (RMST) between treated and control subjects, and (ii) the difference in survival probabilities at a specified time horizon between the two groups. As a result, the target estimands differ across the two packages. In the continuous response and treatment setting described in Section 2.5, `ITERF` estimates the quantity $\tau_2(x)$ from Eq. (4), which corresponds to the slope of the treatment effect under a linear model relating the response to treatment. The `grf` package, through its `causal_forest` function, is also capable of estimating this quantity. While there are notable differences in implementation, such as `grf` employing an approximation of the splitting criterion in Eq. (2), the `HET` method in `ITERF` is conceptually similar to `causal_forest`. Conversely, the method `CMB` in `ITERF` is different because it uses the squared error for predicting the response instead of the heterogeneity maximizing split rule. Finally, in the context of maximum effect estimation with continuous response and treatment, as described in Section 2.6, `ITERF` estimates the quantity $\tau_3(x)$ from Eq. (8), which represents the maximum treatment effect under a quadratic model linking response to treatment. The `grf` package does not currently offer functionality for this type of estimation.

Appendix D reports computing time comparisons between methods implemented in the packages `ITERF`, `randomForestSRC`, and `grf`.

3. Simulation study

Simulation studies evaluating the methods presented in Sections 2.4 and 2.5 are provided in [12,13]. In this section, we conduct a simulation study to illustrate the potential of the new methods for estimating the maximum treatment effect, as introduced in Section 2.6.

3.1. Models

The goal of this simulation study is to evaluate the potential of the random forest methods and verify they work as expected. Three methods are compared. The first two methods are the ones introduced in Section 2.6, `HETMaxQuad` and `CMBMaxQuad`, and are the focus of the study. We use 100 trees in each forest, `mtry=p`, `nodesize=15`, and `obsUniqueInTree=TRUE` for predictions.

We include a simple benchmark model acting as a comparison point. It is the following regression model:

$$Y = \beta_0 + \beta_1 G + \beta_2 G^2 + \sum_{j=1}^{5} \beta_{1j} X_j + \sum_{j=1}^{5} \beta_{2j} X_j G + \sum_{j=1}^{5} \beta_{3j} X_j G^2 + \epsilon.$$

The benchmark model includes linear effects for the covariates, as well as linear and quadratic terms for the treatment, along with their interactions. It has the form (7).

3.2. Data generating processes

There are four data-generating processes (DGPs). The first three are inspired by the setups used in [13]. In all cases, we consider five covariates, $X_1, \ldots, X_5$, which are independently and uniformly distributed over the interval $[0, 10]$. The treatment is uniformly distributed in the interval $[0, 1]$ and independent of the covariates.

DGP 1: Quadratic

This is the base DGP as it follows the form specified in (7). The response Y depends only on the first four covariates and is generated according to the model

$$Y = h(G, X_1, \ldots, X_4) + \epsilon \tag{12}$$

where the ϵ's are independent random errors from a normal distribution with mean 0 and standard deviation 0.5. The function h is given by

$$\begin{aligned} h(G, X_1, \ldots, X_4) =& I(X_1 \le 5 \text{ and } X_3 \le 5) + 2I(X_1 \le 5 \text{ and } X_3 > 5) \\ &+ 3I(X_1 > 5 \text{ and } X_2 \le 5) + 4I(X_1 > 5 \text{ and } X_2 > 5) \\ &+ g_1(X_4)G + g_2(X_4)G^2, \end{aligned} \tag{13}$$

where $g_1(X_4) = (X_4 - 5)^2/5$, $g_2(X_4) = -g_1(X_4)(1 + (X_4 - 5)/10)$, and I is the indicator function. For this DGP, the maximum effect is achieved at a treatment value in the interval $[1/3, 1]$, that varies from one subject to another.

DGP 2: Linear

This is a simple modification of DGP quadratic where we set $g_2(X_4) = 0$ in (13). Hence, h is a linear function. Technically, it still has the form (7) but with $\beta_2(x) = 0$. For this DGP, the maximum effect is always achieved at a treatment value of 1, which is the maximum possible treatment value.

DGP 3: Piecewise linear

This DGP is not of the form (7). It is similar to DGP quadratic but with a piecewise linear (broken-stick) shape instead of a quadratic one. The h function is given by

$$\begin{aligned} h(G, X_1, \ldots, X_4) =& I(X_1 \le 5 \text{ and } X_3 \le 5) + 2I(X_1 \le 5 \text{ and } X_3 > 5) \\ &+ 3I(X_1 > 5 \text{ and } X_2 \le 5) + 4I(X_1 > 5 \text{ and } X_2 > 5) \\ &+ g_1(X_4)GI(G \le g_3(X_4)) \\ &+ (g_1(X_4)g_3(X_4) - (G - g_3(X_4))/2)I(G > g_3(X_4)), \end{aligned}$$

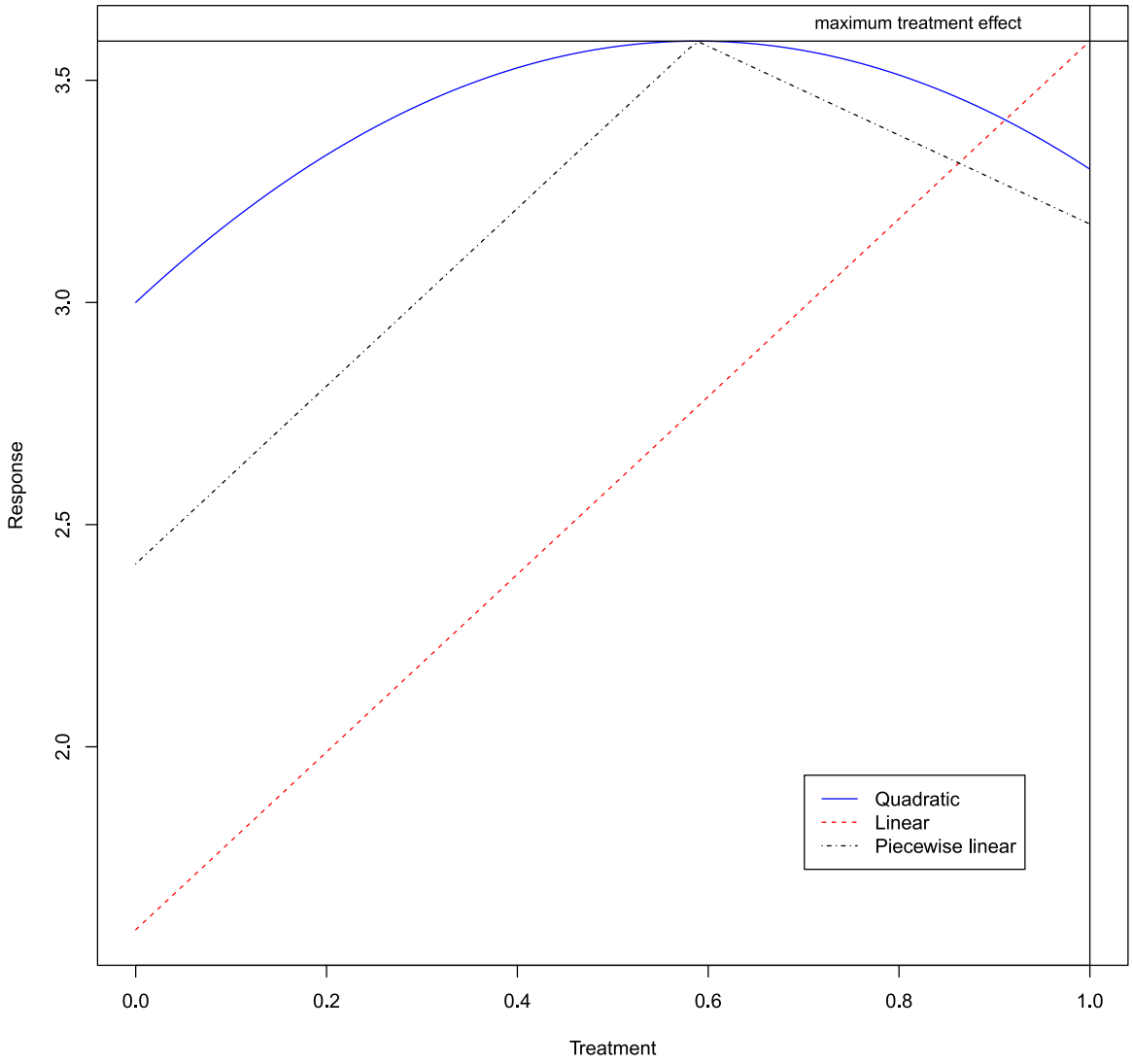

Fig. 1. Illustration of the treatment effect for DGP quadratic, DGP linear and DGP piecewise linear.

(14)

where $g_3(X_4) = 5/(5 + X_4)$. As for DGP quadratic, the maximum effect is achieved at a treatment value in the interval $[1/3, 1]$, that varies from one subject to another. To fix ideas, Fig. 1 illustrates typical treatment effect functions for DGP quadratic, linear and piecewise linear. The maximum effect is the same for the three examples depicted.

DGP 4: Bench

To be fair with the benchmark model, we also include DGP bench which has the same specification as the benchmark model. As such, the benchmark model will naturally perform the best. However, it is still informative to assess how closely the random forests can approach this optimal performance. For this DGP, Y is generated as

$$\begin{aligned} Y =& X_1 + X_2 + X_3 + X_4 + X_5 + X_1G + X_2G + X_3G + X_4G + X_5G \\ & - .5X_1G^2 - .6X_2G^2 - .7X_3G^2 - .8X_4G^2 - .9X_5G^2 + G - .5G^2 + e \end{aligned} \tag{15}$$

where the ϵ's are independent random errors from a normal distribution with mean 0 and standard deviation 1.

Additional noise covariates

Variants of the four DGPs described above where we add 15 additional independent noise covariates are also considered in the study.

3.3. Sample sizes and evaluation criteria

We consider ten training sample sizes given by $n = k^2$ for $k = 10, 20, \ldots, 100$. For each sample size, 100 simulation runs are performed. In each run, a separate test sample of size 500 is used to evaluate model performance.

Consequently, we consider 80 scenarios: 4 DGPs × 10 training sample sizes × 2 (with and without additional noise covariates).

We use four metrics to evaluate the performance. The first one is the root mean squared error (RMSE), defined as:

$$\text{RMSE} = \left(\frac{1}{ntest} \sum_{i=1}^{ntest} (\hat{\tau}_{3i} - \tau_{3i})^2 \right)^{1/2}, \tag{16}$$

where $\hat{\tau}_{3i}$ and τ_{3i} are the estimated and true values of the maximum effect, as defined in (8), in the test set. Smaller values of RMSE indicate a better performance.

The second one is the mean absolute percentage error (MAPE) given by

$$\text{MAPE} = \frac{1}{ntest} \sum_{i=1}^{ntest} 100 \frac{|\hat{\tau}_{3i} - \tau_{3i}|}{\tau_{3i}}. \tag{17}$$

Smaller values of MAPE indicate a better performance.

The first two metrics assess how close the estimated values are to the true ones. The third metric is a ranking metric that evaluates whether

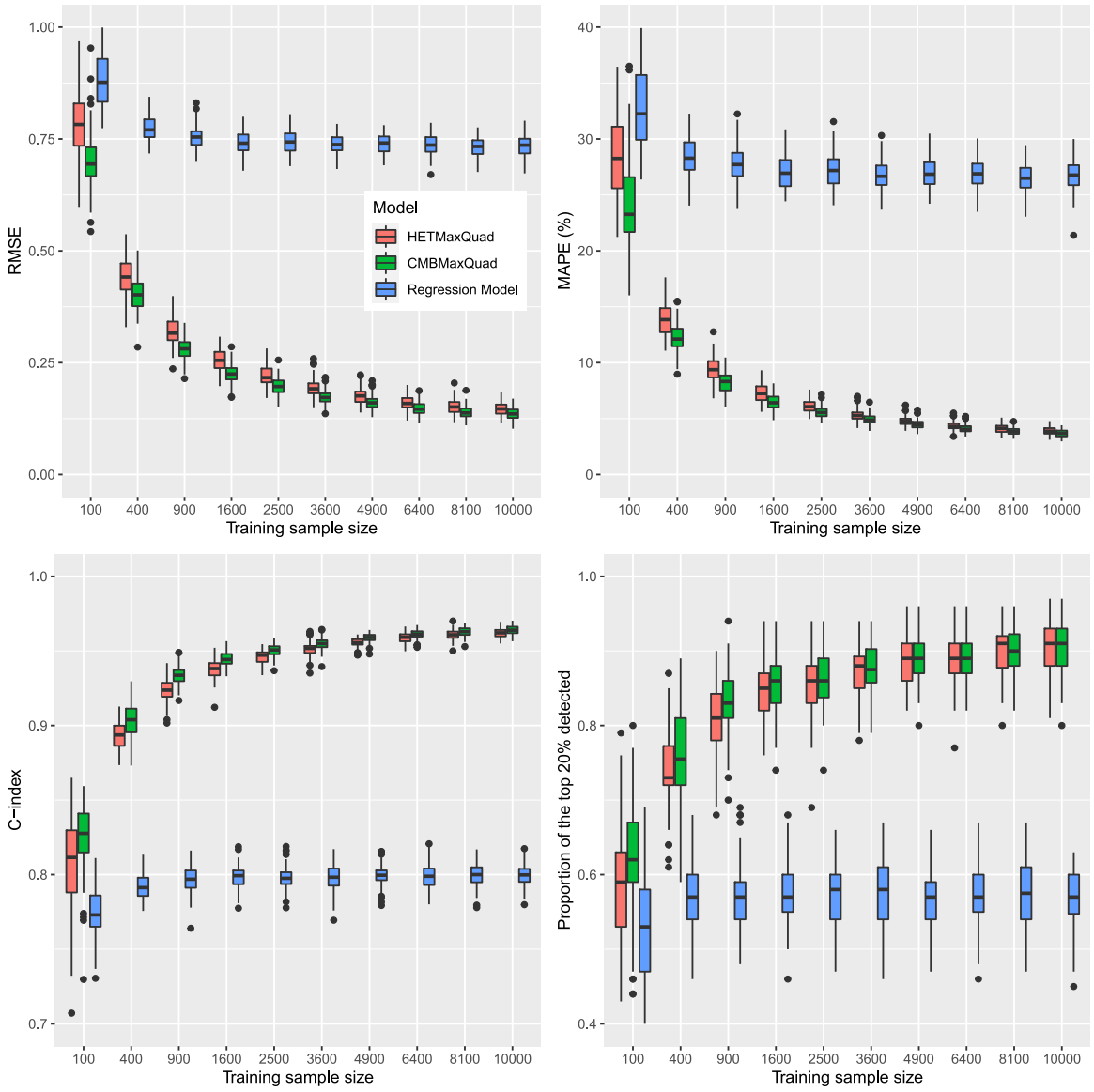

Fig. 2. Results for DGP quadratic with no additional noise covariates.

the model correctly orders subjects according to their true treatment effects. For this purpose, we use the concordance index (C-index), one of the most widely adopted ranking metrics. In our context, the C-index is defined as:

$$\text{C-index} = \frac{2}{ntest(ntest-1)} \sum_{i=1}^{ntest-1} \sum_{j=i+1}^{ntest} (I(\hat{\tau}_{3i} > \hat{\tau}_{3j}) I(\tau_{3i} > \tau_{3j}) + I(\hat{\tau}_{3i} < \hat{\tau}_{3j}) I(\tau_{3i} < \tau_{3j})).$$

The C-index measures the proportion of subject pairs that the model ranks in the correct order with respect to their true treatment effects. Higher values of the C-index indicate better ranking performance.

In many applications, it is of particular interest to identify the individuals who are likely to benefit the most from treatment. In our context, since we aim to estimate the maximum treatment effect, this corresponds to identifying the subjects for whom the treatment has the greatest potential impact. We assume that our goal is to detect the top 20% of the most promising candidates. To operationalize this, we classify as candidates for treatment those subjects whose estimated treatment effects fall within the top 20%. For a given test set, let T_{20} denote the indices of the subjects whose τ_3 values are in the top 20%, and let $\hat{T}_{20}$ represent the indices of the subjects whose $\hat{\tau}_3$ values fall in the top 20%. Let n_{20} denote the number of subjects in each of these sets. The criterion is given by

$$\text{proportion of the top 20\% detected} = \frac{\text{number of subjects that are in both } T_{20} \text{ and } \hat{T}_{20}}{n_{20}}.$$

It measures the model's ability to correctly identify subjects belonging to the true top 20% in terms of maximum treatment effect. Larger values of this criterion indicate a better performance.

3.4. Results

We begin by discussing the results for the scenarios with no additional noise covariates. For all DGPs, the two random forests methods have a similar performance, although `CMBMaxQuad` has a slightly better performance than `HETMaxQuad` in most cases.

The results for DGP quadratic are presented in Fig. 2 and align with expectations. Each plot corresponds to one evaluation criterion, with boxplots illustrating the distribution of the criterion for each method across varying training sample sizes. The random forests consistently outperforms the regression model. As the sample size increases, the

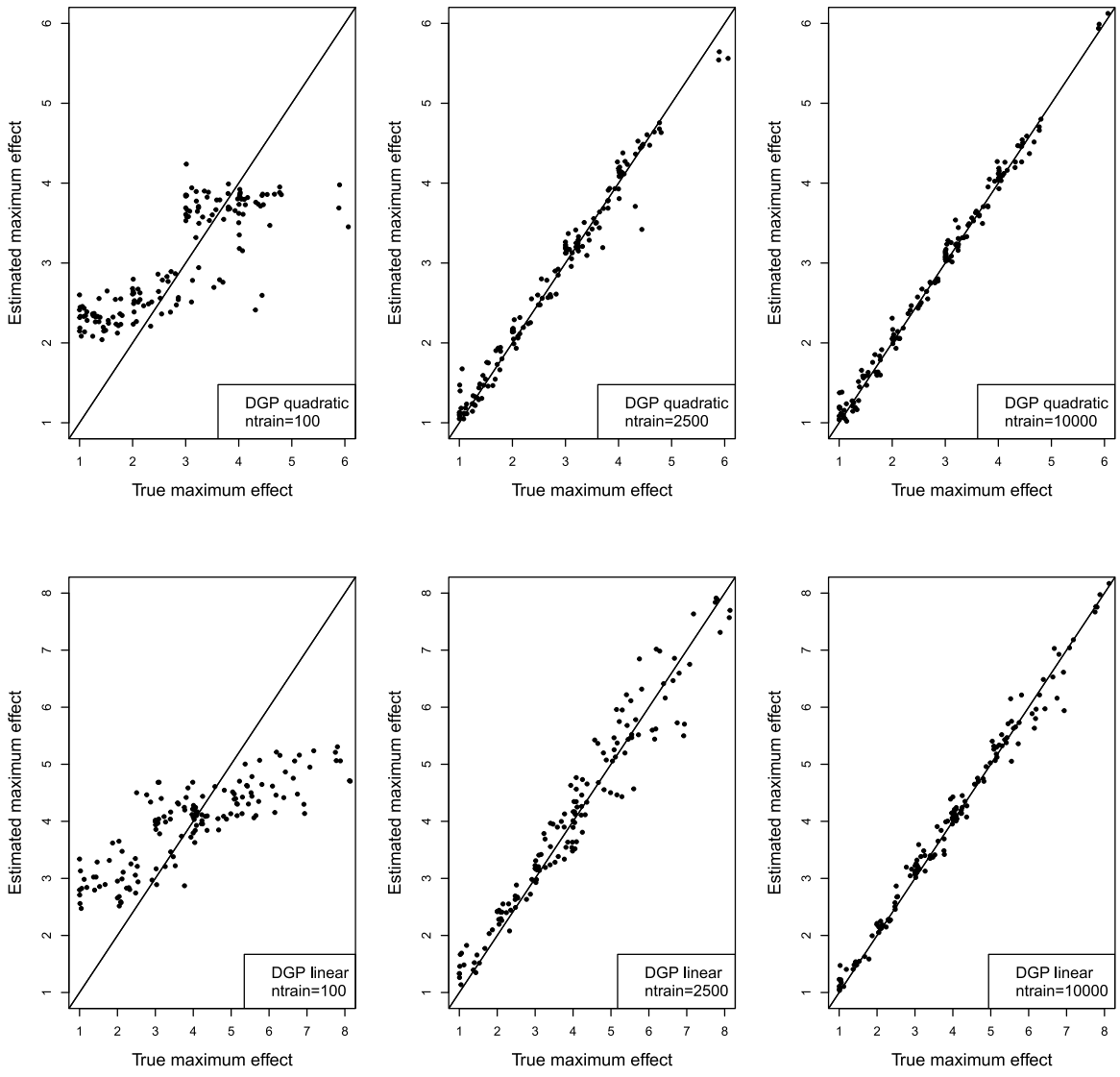

Fig. 3. True vs estimated values for DGP quadratic and DGP linear, for the random forest model `HETMaxQuad`.

performance of the regression model quickly plateaus, limited by its structural misspecification, while the random forests continues to improve, benefiting from its flexibility and the increasing availability of data. An illustration of `HETMaxQuad` model's fit quality is provided in the top panel of Fig. 3, which displays estimated versus true values for representative test samples of sizes 100, 2500, and 10,000. For $ntrain = 100$ (left plot), the fit is relatively poor, though a tendency to correctly rank subjects is already visible. The fit improves substantially for $ntrain = 2,500$ (middle plot) and becomes even more accurate at $ntrain = 10{,}000$ (right plot). The fit quality plots for `CMBMaxQuad` are similar and not shown.

The results for DGP linear are shown in Fig. 4. For this DGP, the maximum effect is always achieved at the boundary value of $G = 1$. The progression of the performance criteria closely mirrors that of the DGP quadratic, indicating that the random forests are also well-suited to this setting. The lower panel of Fig. 3 presents the estimated versus true values for `HETMaxQuad`, leading to conclusions similar to those drawn for DGP quadratic.

Fig. 5 presents the results for DGP piecewise linear. Unlike the previous two DGPs, this one does not follow the form given in (7). As a result, the within-node model used by the forests is misspecified. Nevertheless, since a quadratic model can reasonably approximate the piecewise linear function used (which includes a single knot), the random forests still yield accurate estimates. This is confirmed by the upper panel of Fig. 6, which shows that the estimations are quite precise.

The DGP benchmark results are displayed in Fig. 7. As expected, the regression model outperforms the random forests due to its correct specification. However, the random forests still demonstrates consistent improvement with increasing training sample size. These improvements in fit quality are also evident in the lower panel of Fig. 6.

The main conclusions for the variants with 15 additional noise covariates are as follows: while these noise covariates slightly degrade performance at smaller training sample sizes, their impact diminishes rapidly as the sample size grows. This is illustrated in Fig. 8, which compares the performance of `CMBMaxQuad` with and without the additional noise covariates for DGP quadratic. As shown, the impact is noticeable at $ntrain = 100$, but becomes negligible as the training size increases. A similar trend is observed across the other DGPs. The same patterns occur for `HETMaxQuad`. As for the scenarios with no additional noise covariates, `CMBMaxQuad` has a slightly better performance than `HETMaxQuad` in most cases.

4. Real data example

This example explores the relationship between sleep duration and cognitive health in the elderly and is motivated by [18], which offers

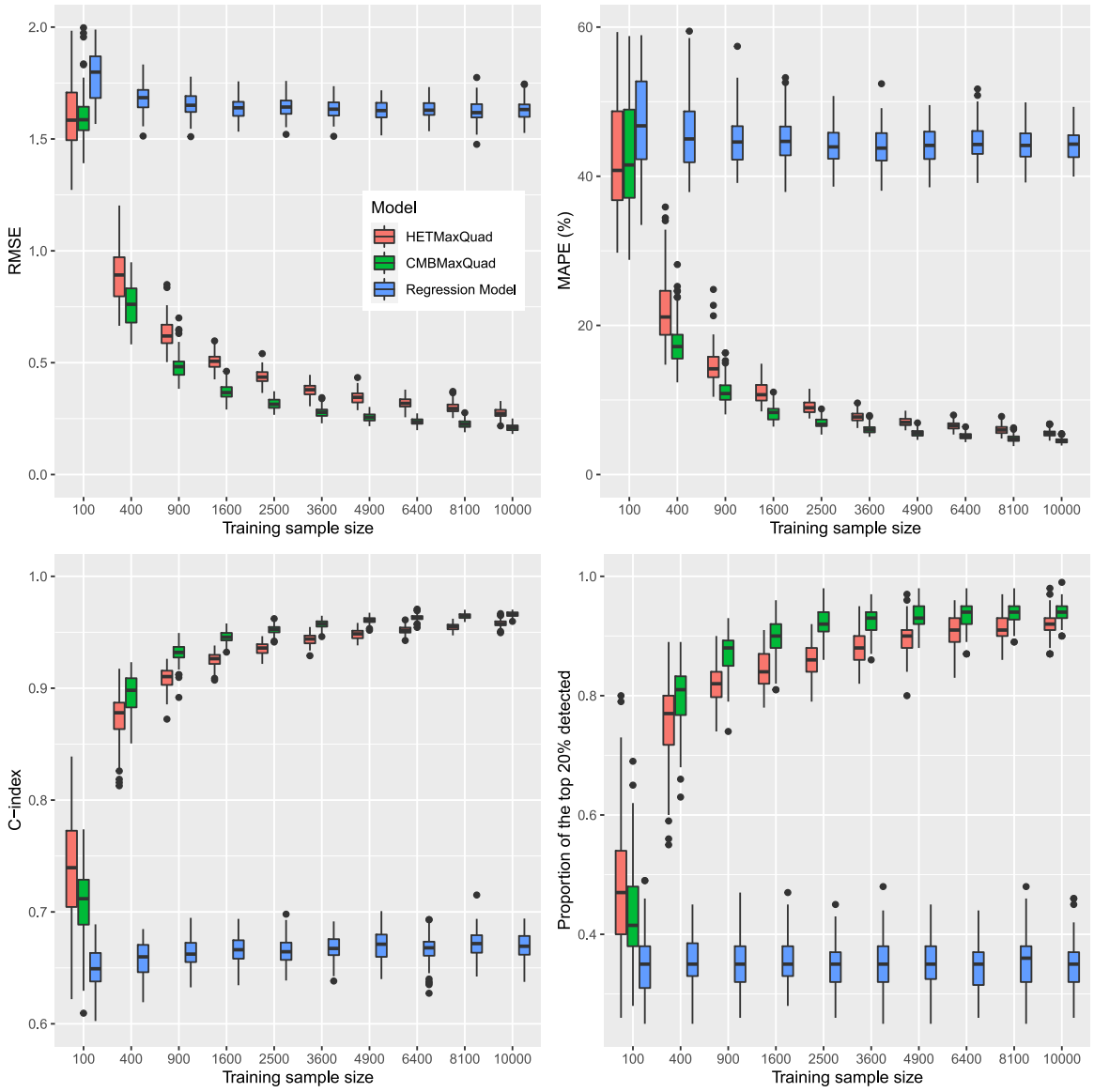

Fig. 4. Results for DGP linear with no additional noise covariates.

more background and relevant references. In their study, this relationship is analyzed at the population level, with limited subgroup results reported. Here, we extend the investigation to the individual level.

Following the selection and exclusion criteria outlined in Figure 1 of [18], we extract data from NHANES [19] for the years 2011–2014 using the R package `nhanesA` [20]. Appendix E.1 provides details about the extracted data. The initial sample includes 2928 observations, three less than what is reported in [18].

The response variable is cognitive function (CF). As explained in [18], CF is computed as the average score across several objective tests assessing different cognitive domains. Appendix E.2 explains in detail how we compute it. The treatment (exposure) variable is self-reported sleep duration (SD), measured in hours.

Covariates include nine demographic, socioeconomic, lifestyle, and health-related factors: gender, race, age, education level, poverty income ratio (PIR), waist circumference, body mass index (BMI), physical activity, and diabetes. After removing observations with missing covariate values, the final sample comprises 2515 observations. Descriptive statistics are provided in Appendix E.3.

Alcohol frequency, smoking status, and depressive symptoms are also included as covariates in [18]. However, these variables have substantial missing data; excluding them would reduce the sample size to 791. For this reason, we omit them here. However, additional sensitivity analysis could consider using imputations methods to include them.

Figure 3 in [18] shows that there is an inverted U-shape relationship between SD and CF at the population level. To verify whether this pattern holds in our data, we first fit a generalized additive model (GAM) with a smooth term for SD and linear terms for the nine covariates, using the R package `mgcv` [21]. The black curve in Fig. 9 shows the estimated smooth effect of SD, along with 95% confidence bands. This curve closely resembles the one reported in Figure 3 of [18]. Next, we replace the smooth term for SD with a quadratic specification, modeling SD as $\beta_1 \mathrm{SD} + \beta_2 \mathrm{SD}^2$. This reduces the model to a standard linear regression. The red curve in Fig. 9 depicts the fitted quadratic effect, which provides a reasonable approximation of the SD effect. This observation motivates the use of the proposed random forest methods `HETMaxQuad` and `CMBMaxQuad`.

Fig. 18 in Appendix E.3 displays the distribution of SD. For the subsequent analysis, due to their low frequencies, values of SD below 4 are recoded to 4, and values above 10 are recoded to 10.

The previous models considered a population-level effect for sleep duration. Here, we allow for individual-level effects using the proposed

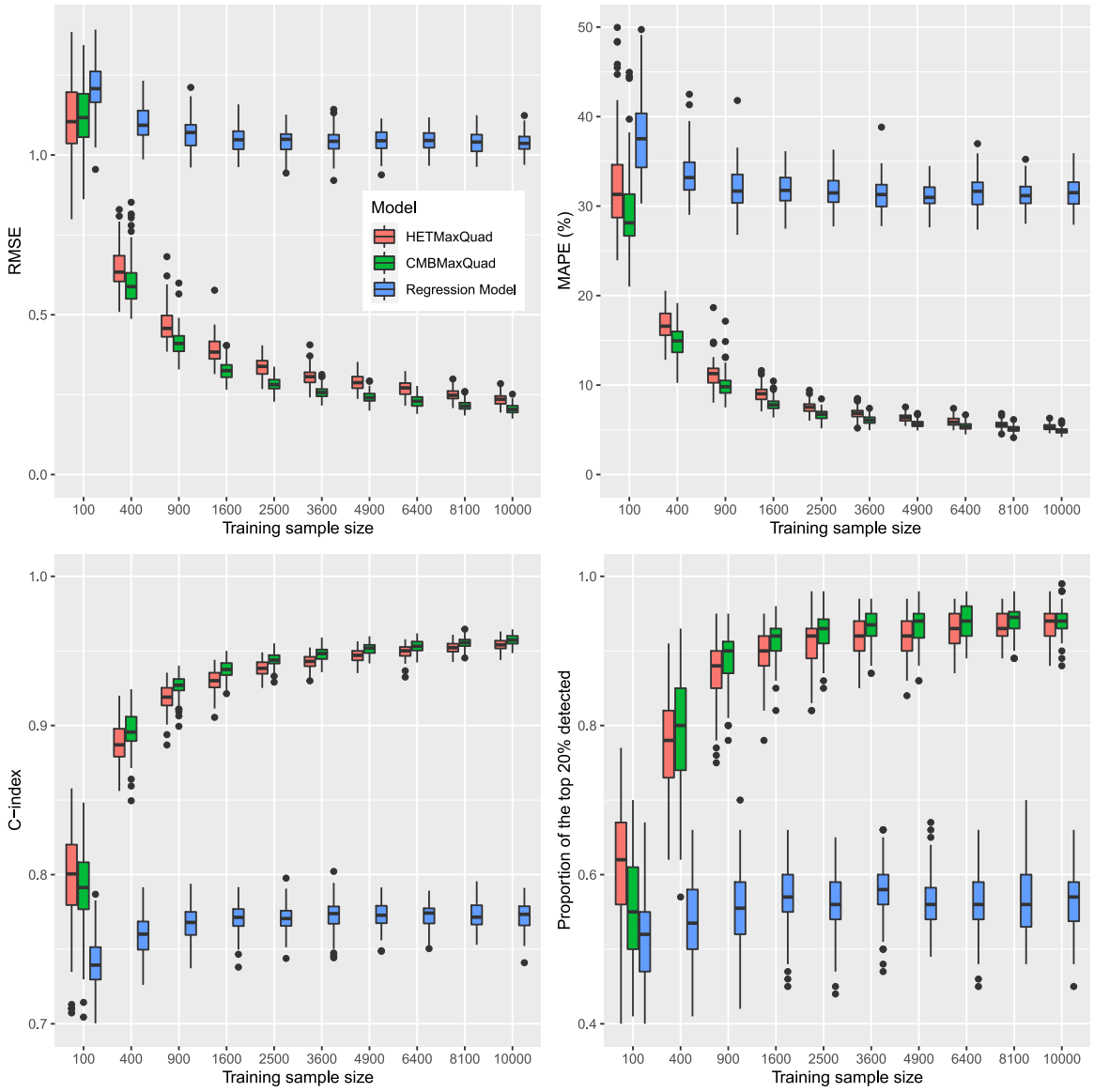

Fig. 5. Results for DGP piecewise linear with no additional noise covariates.

method `HETMaxQuad` to estimate the maximum treatment effect, with CF as the response, SD as the treatment and gender, race, age, education level, PIR, waist circumference, BMI, physical activity and diabetes as covariates. We also applied `CMBMaxQuad` but the results are very similar and not discussed further (see Fig. 28 in Appendix E.4 that compares the estimated values obtained from the two forests). The random forest is built with 500 trees, `mtry=9` and `nodesize=100`. The OOB BOP is used to obtain the estimated treatment effects for all 2515 observations.

In this context, the estimated maximum effect from `HETMaxQuad` represents the maximum expected value of CF achievable with the optimal amount of sleep (i.e., the best value of SD for that individual). Importantly, this is not a prediction of CF given the observed SD and covariates; rather, it is the expected CF if the subject had the optimal SD. The distribution of these estimated maximum treatment effects is shown in Fig. 10. Notably, a subgroup of subjects exhibits small estimated maximum effects, primarily associated with low education levels, as discussed below.

Next, we compute variable importance scores (VIMP) for the covariates using the fit-the-fit approach [22–25]. Specifically, we use the estimated maximum treatment effect from `HETMaxQuad` as the dependent variable and fit another random forest with the same nine covariates. VIMP values are extracted from this forest and presented in Fig. 11. `Education` and `Age` emerge as the most important variables.

We then examine these two covariates in greater detail. Fig. 12 illustrates how the estimated maximum treatment effect varies by education level: higher education is associated with greater achievable cognitive function, while subjects with less than a 9th grade education exhibit notably lower values. Fig. 13 shows variation by age: the maximum achievable CF remains stable until approximately age 72, declines until about 77, and then stabilizes again. Fig. 14 explores the interaction between age and education. The general trends observed in Figs. 12 and 13 persist, but this joint analysis provides a more nuanced interpretation. The impact of age is more pronounced for individuals with higher education levels (high school graduates or above). These individuals reach a higher maximum achievable CF in their 60s but experience a steeper decline when the negative effect of age emerges in their 70s.

5. Discussion

The methods available in the `ITERF` package provide fast and user-friendly estimations for various measures of treatment effect, notably

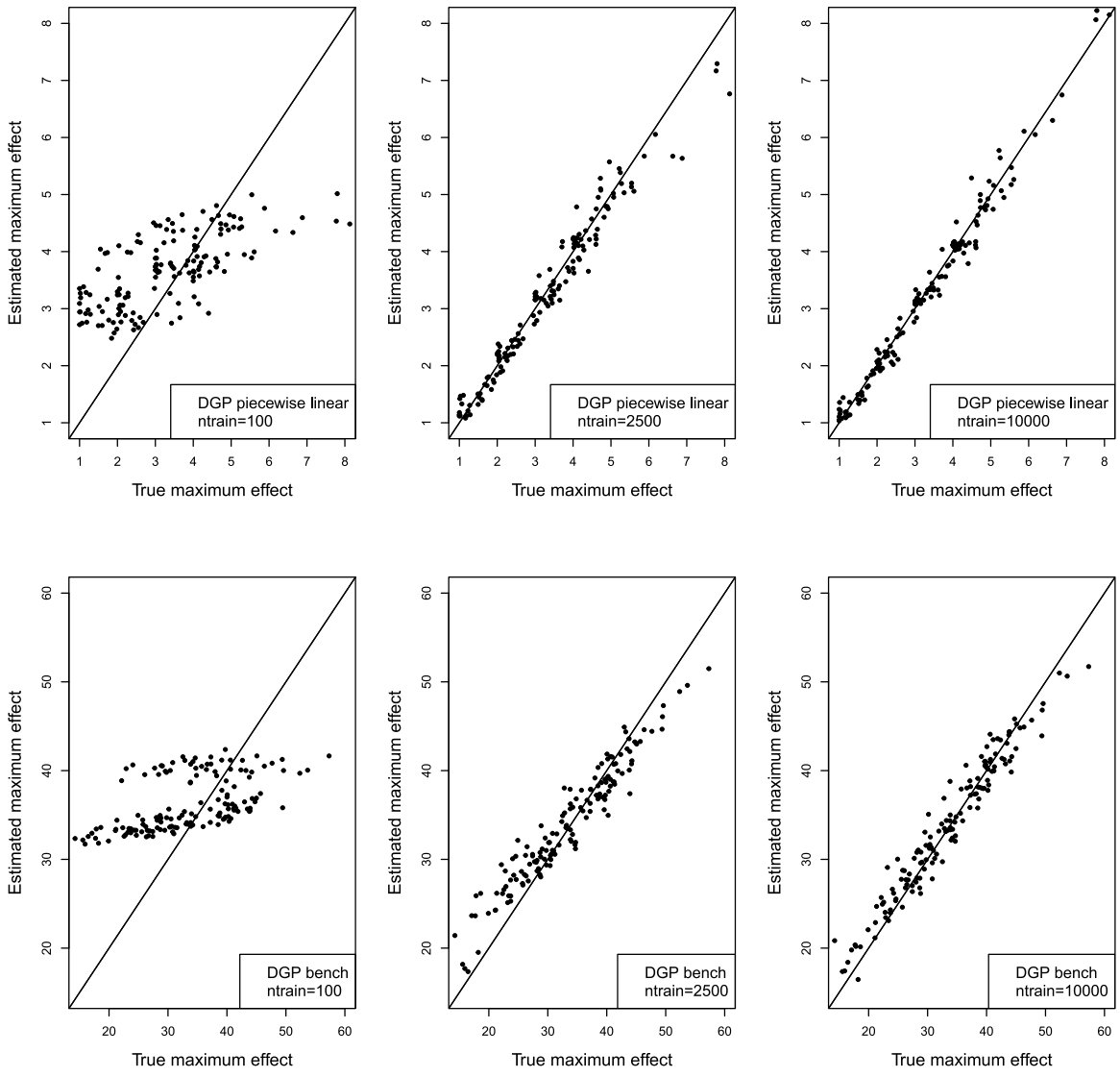

Fig. 6. True vs estimated values for DGP piecewise linear and DGP bench, for the random forest model `HETMaxQuad`.

the ones proposed in [12,13] for a survival response and a binary treatment and for a continuous response and a continuous treatment. In addition, two new methods to estimate the maximum treatment effect are proposed and included in `ITERF`. A simulation study demonstrates that these methods perform as expected and shows considerable promise. A real data example illustrates the use of these new methods for exploring the link between sleep duration and cognitive health in the elderly.

The current implementation of the methods provide point estimates of the target treatment effects. However, in many applications, it is important to assess the precision of these estimates, and constructing confidence intervals is a standard approach for this purpose. Confidence intervals for random forests has been explored in prior work [26–28]. For instance, [28] investigates using the delete-d jackknife method to compute confidence intervals for variables importance measures. Preliminary investigations indicate that this approach cannot be directly applied in our context, particularly in the setting of maximum effect estimation, where the estimand involves the maximum of a function. In such cases, the convergence rate of the estimators deviates from the usual $\sqrt{n}$ rate. As a result, confidence intervals constructed using the standard delete-d jackknife method fail to achieve nominal coverage. While corrections are theoretically possible, they require precise knowledge of the estimators' convergence rates, which presents a complex challenge beyond the scope of this article. Therefore, we defer the development of valid confidence interval estimation procedures to future work. Nonetheless, we recognize that incorporating such functionality into the package would significantly enhance its practical utility.

Future work could further explore the method introduced for estimating the maximum treatment effect. In this setting, the working assumption is that the relationship between the response and the treatment follows a quadratic form. Simulation results based on a piecewise linear DGP indicate that the method maintains satisfactory performance even under moderate deviations from this assumption. Nonetheless, a more comprehensive investigation into the robustness of the method under varying degrees of model misspecification would be valuable. In addition, investigating alternatives to the quadratic model, such as using piecewise linear or spline-based models, could offer greater flexibility and potentially improved performance.

The current version of the package does not include dedicated functionality for handling missing data. In practice, standard approaches such as multiple imputation can be employed to address this issue prior to model fitting. However, integrating native tools for managing missing data within `ITERF` would enhance usability. Another promising direction for future development involves extending the package to accommodate discrete and multi-class response and treatment variables. Such functionality would broaden the applicability of `ITERF` to a wider range of empirical settings and research questions.

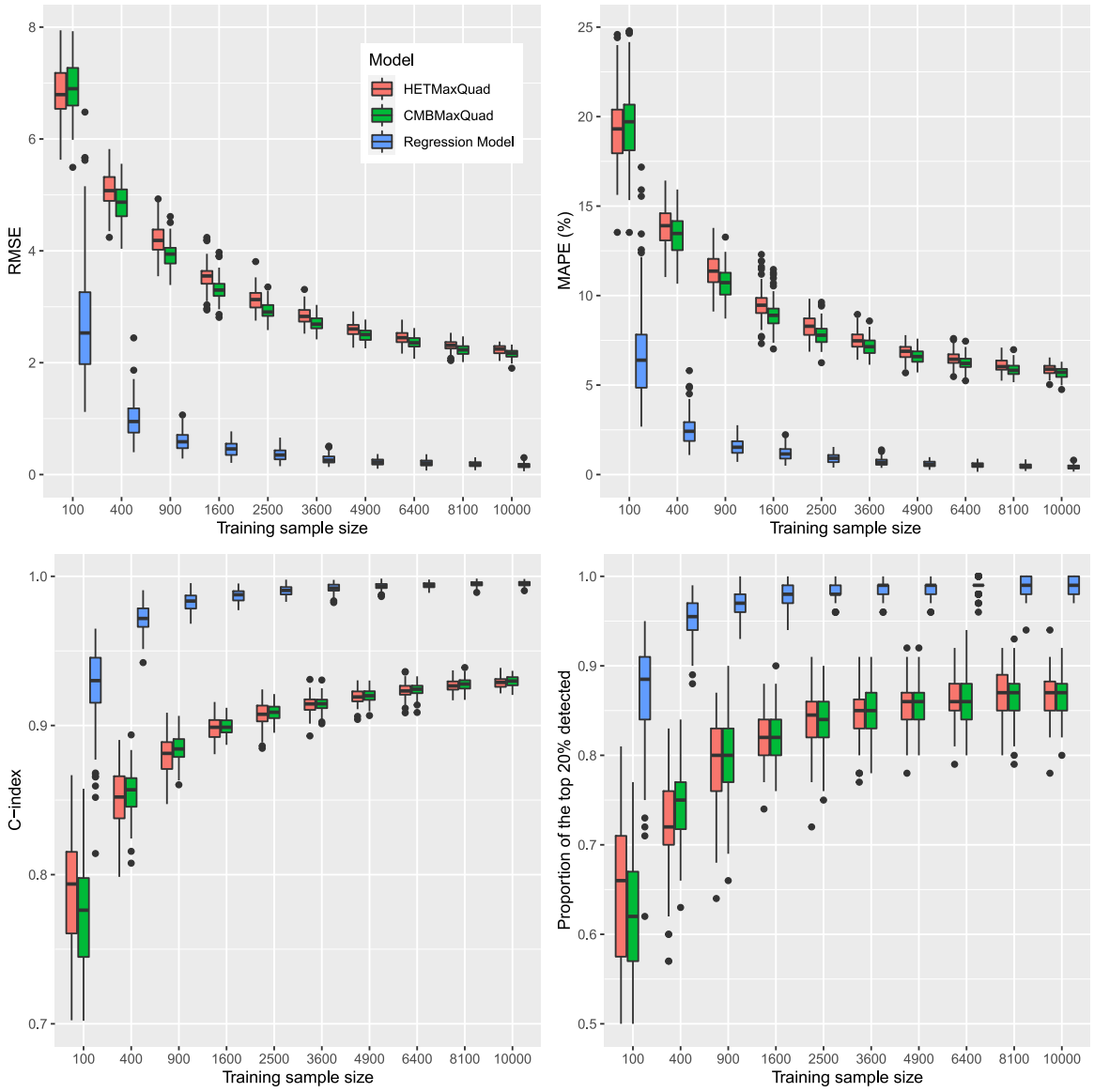

Fig. 7. Results for DGP bench with no additional noise covariates.

CRediT authorship contribution statement

Sami Tabib: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Conceptualization. **Denis Larocque:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Conceptualization.

Ethics statement

This research does not need an ethics statement. No data collection was performed. The simulations in the article are based on simulated data.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Microsoft Copilot in order to improve the quality of the language of the initial draft. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Funding sources

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We would like to warmly thank the reviewers for their relevant, constructive and helpful comments that paved the way to an improved version of the article.

Appendix A. Assumptions

We work under the potential outcomes framework [3], which require assumptions of positivity and unconfoundedness. The first one

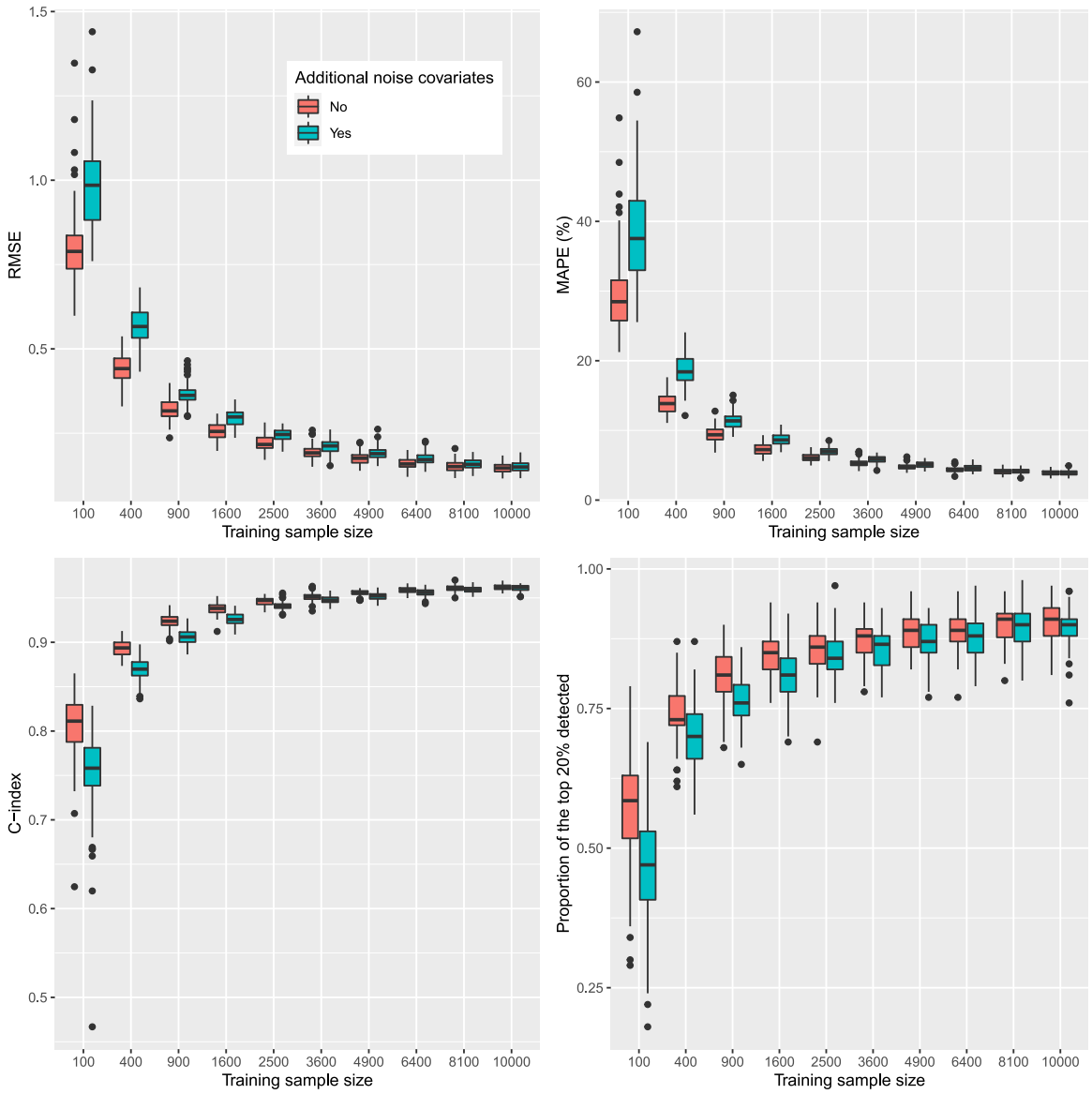

Fig. 8. Results for DGP quadratic with and without additional noise covariates, for the `CMBMaxQuad` method.

states that for any given covariates, all treatment values are possible. The second assumption states that all confounders are included in X, meaning that, conditional on the covariates, the treatment value is independent of the potential outcomes (one for each possible value of the treatment). Here are more details for each setting considered.

In all cases, let $f_X(x)$ be the density of X. With a binary treatment, the positivity assumption requires that $P(G = g|x) > 0$, for $g = 0, 1$ and for all x such that $f_X(x) > 0$. With a continuous treatment, the positivity assumption requires that $f_{G|X}(g|x) > 0$, for all g and for all x such that $f_X(x) > 0$, where $f_{G|X}$ is the conditional density of G given X. Diagnostics for the positivity assumption for a binary and a continuous treatment are proposed in [29,30], respectively.

Survival Data with a Binary Treatment (Section 2.4)

In this case, Y is a survival time with possible censoring and G is binary. In addition to the positivity assumption for a binary treatment, we also assume that the censoring time and the event time are independent, given the covariates, in each treatment group.

Slope Estimation with a Continuous Response and Treatment (Section 2.5)

In this case, Y and G are continuous. In addition to the positivity assumption for a continuous treatment, we also assume

$$E[Y|G = g, X = x] = \mu(x) + \tau_2(x)g$$

Maximum Effect Estimation with a Continuous Response and Treatment (Section 2.6)

In this case, Y and G are again continuous. In addition to the positivity assumption for a continuous treatment, we also assume

$$E[Y|G = g, X = x] = \beta_0(x) + \beta_1(x)g + \beta_2(x)g^2.$$

Appendix B. Code examples

In this appendix, we provide code examples for the random forest methods available in the package `ITERF`. Additional examples with real data are available in the function descriptions in the package.

B.1. Estimation of the maximum treatment effect for a continuous response and a continuous treatment with the function `HETMaxQuad`

In this example, we estimate the maximum treatment effect $\tau_3(x)$ defined in Section 2.6. We show how to compute estimations with the

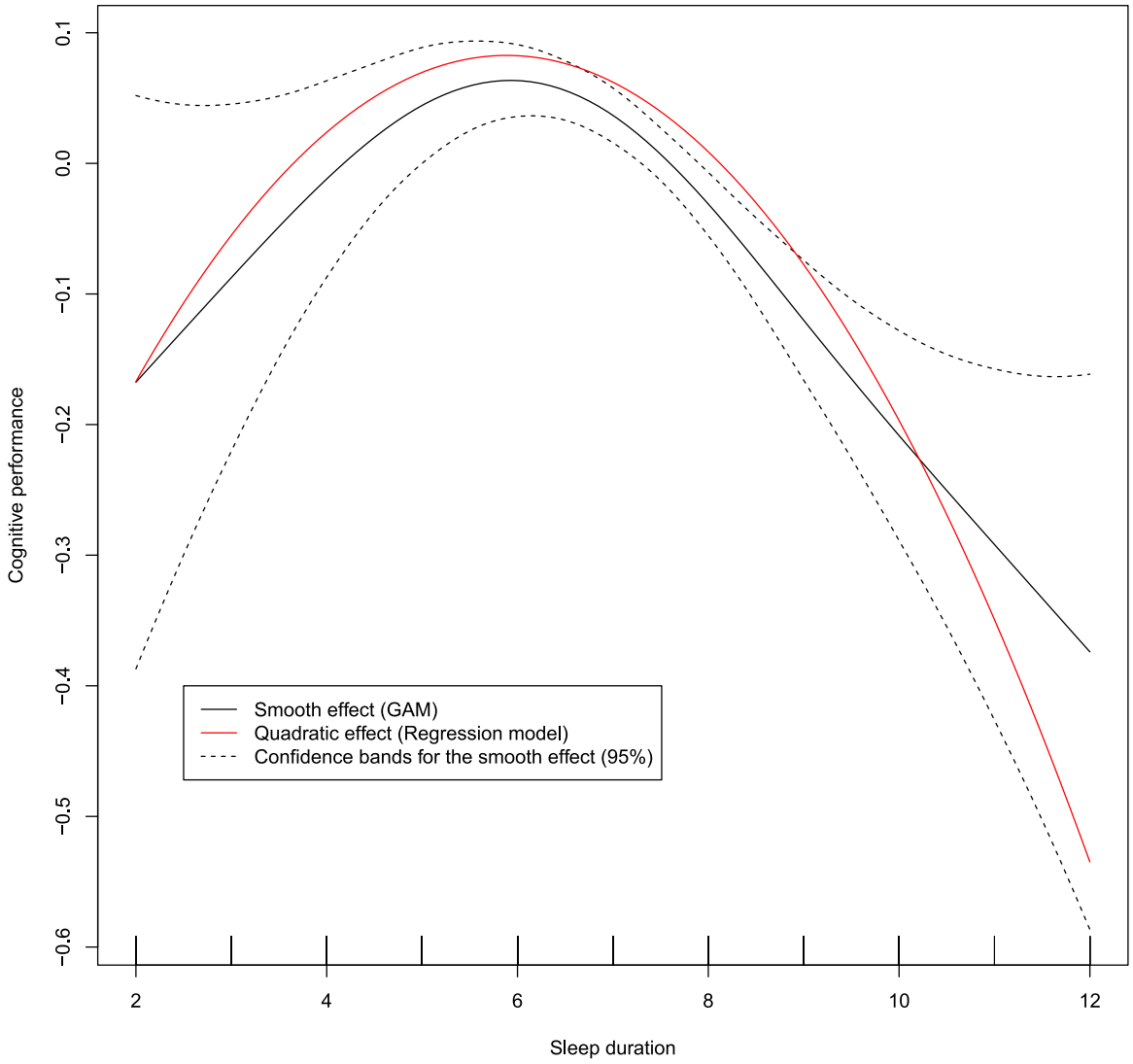

Fig. 9. Sleep duration effect (population level) on cognitive performance for a generalized additive model (GAM) and for a linear regression model with a quadratic effect.

three variants described in Section 2.7. We also show how to select the number of cores for parallel computing. The function `generatedatatau3` generates data according to DGP quadratic described in Section 3.2. We use the function `HETMaxQuad` but all these examples can also be executed with the function `CMBMaxQuad`.

```
# Function to generate data
generatedatatau3=function(n)
{
# INPUT:
# n =sample size
# OUTPUT:
# y = response
# g = treatment value (continuous)
# x1,...,x5 = covariates
# tau3 = Maximum treatment effect

x1 = runif(n)*10
x2 = runif(n)*10
x3 = runif(n)*10
x4 = runif(n)*10
x5 = runif(n)*10
g = runif(n)
b0=(x1<=5)*(x3<=5)*1 + (x1<=5)*(x3>5)*2 + (x1>5)
    *(x2<=5)*3 +
   (x1>5)*(x2>5)*4
b1=((x4-5)^2)/5

b2=-b1*(1+(x4-5)/10)
y=b0 + g*b1 + b2*g^2 + rnorm(n,0,.5)
truemax=-b1/(2*b2)
tau3=b0+b1*truemax+b2*truemax^2
dat = data.frame(y,g,x1,x2,x3,x4,x5,tau3)
dat
}

library(ITERF)

# Training and test sample sizes
ntrain=3000
ntest=200

# Data generation
set.seed(345)
datrain=generatedatatau3(ntrain)
datest=generatedatatau3(ntest)

# Fit the random forest
```

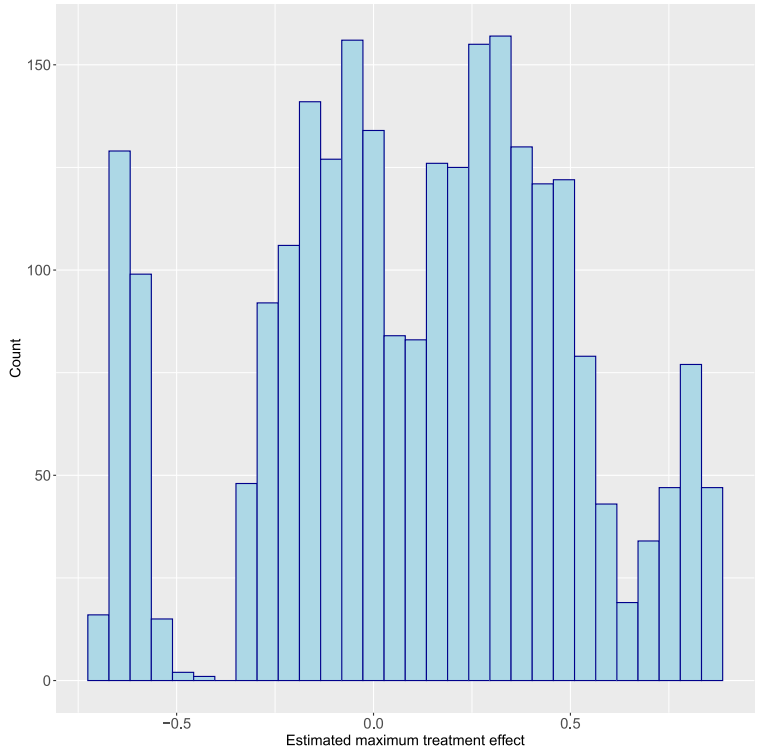

Fig. 10. Histogram of the estimated maximum treatment effect for the model `HETMaxQuad`.

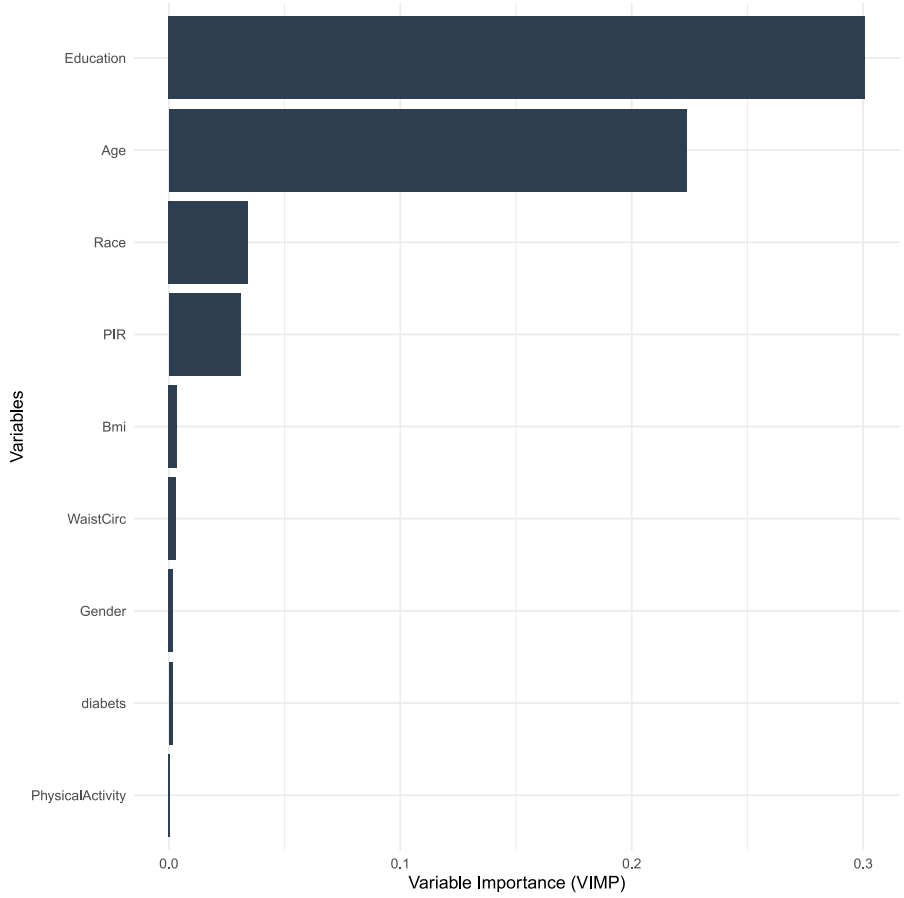

Fig. 11. Variable importance (VIMP) for the estimated maximum treatment effects obtained from the model `HETMaxQuad`.

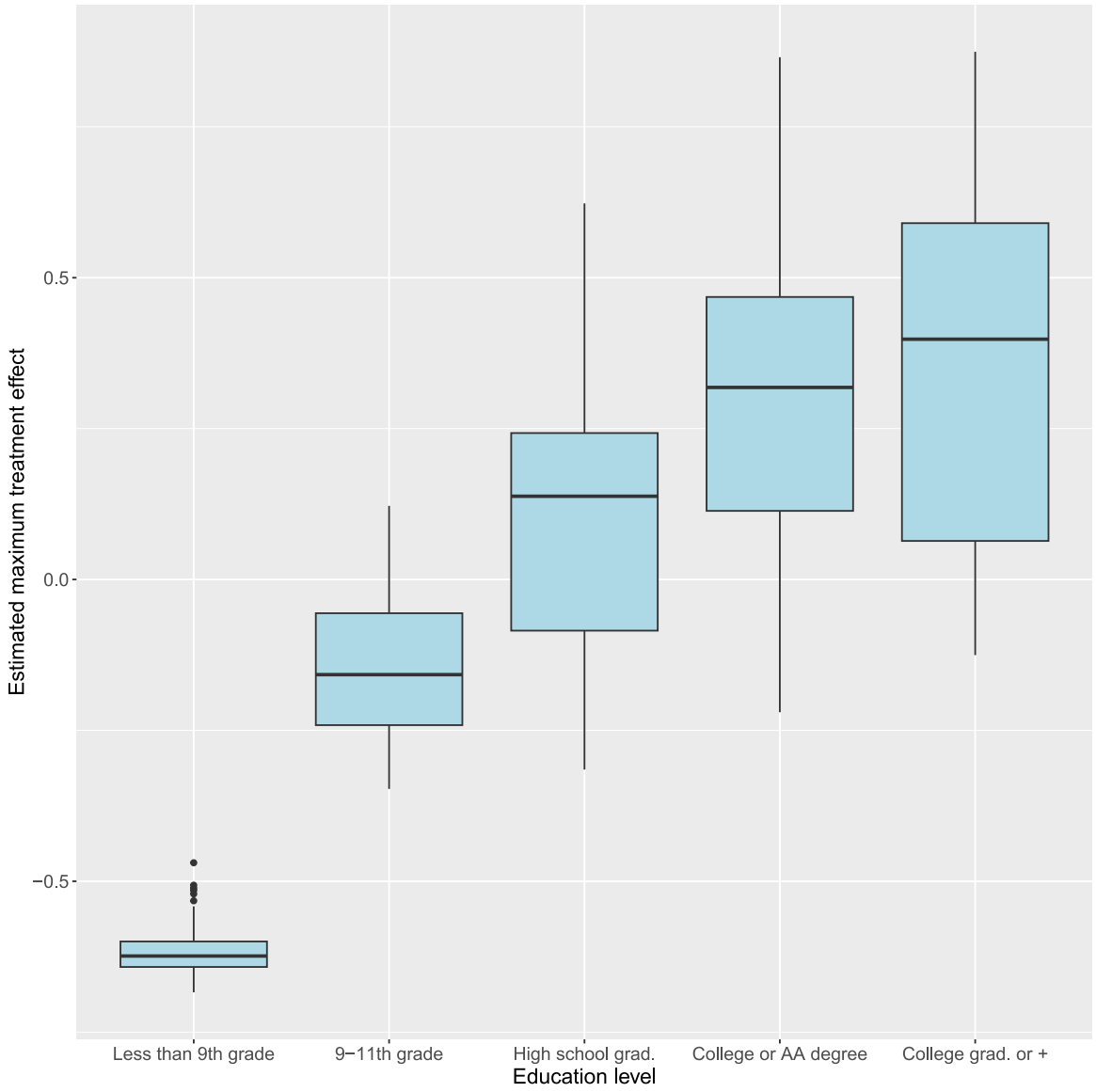

Fig. 12. Distribution of estimated maximum treatment effects at each education level.

```
mod = HETMaxQuad(f(y, g) ~ x1+x2+x3+x4+x5, data
    = datrain ,ntree = 100,
  samptype ="swr",nodesize=15,mtry=5)

# Compute OOB estimations for test data
predoob = predict(mod, newdata=datest,
  OOB = TRUE, obsUniqueInTree = TRUE)$predicted
    [,2]

# Compute estimations with the unique In-Bag
    method for test data
preduniqueinbag = predict(mod, newdata=datest,
  OOB = FALSE, obsUniqueInTree = TRUE)$predicted
    [,2]

# Compute estimations with the all In-Bag method
     for test data
predallinbag = predict(mod, newdata=datest,
  OOB = FALSE, obsUniqueInTree = FALSE)$
    predicted[,2]

# Compute estimations with the OOB method for
    train data
predtrain = predict(mod, data=datrain,
  OOB = TRUE, obsUniqueInTree = TRUE)$predicted
    [,2]

# Plot true versus estimated values
par(mfrow=c(2,2))

plot(datest$tau3,predoob,xlab="True maximum
    effect",
  ylab="Estimated maximum effect",main="
    HETMaxQuad Method (test set OOB estimations)
    ")
abline(a=0,b=1)

plot(datest$tau3,preduniqueinbag,xlab="True
    maximum effect",
  ylab="Estimated maximum effect",main="
    HETMaxQuad Method (test set unique In-Bag
    estimations)")
abline(a=0,b=1)

plot(datest$tau3,predallinbag,xlab="True maximum
     effect",
```

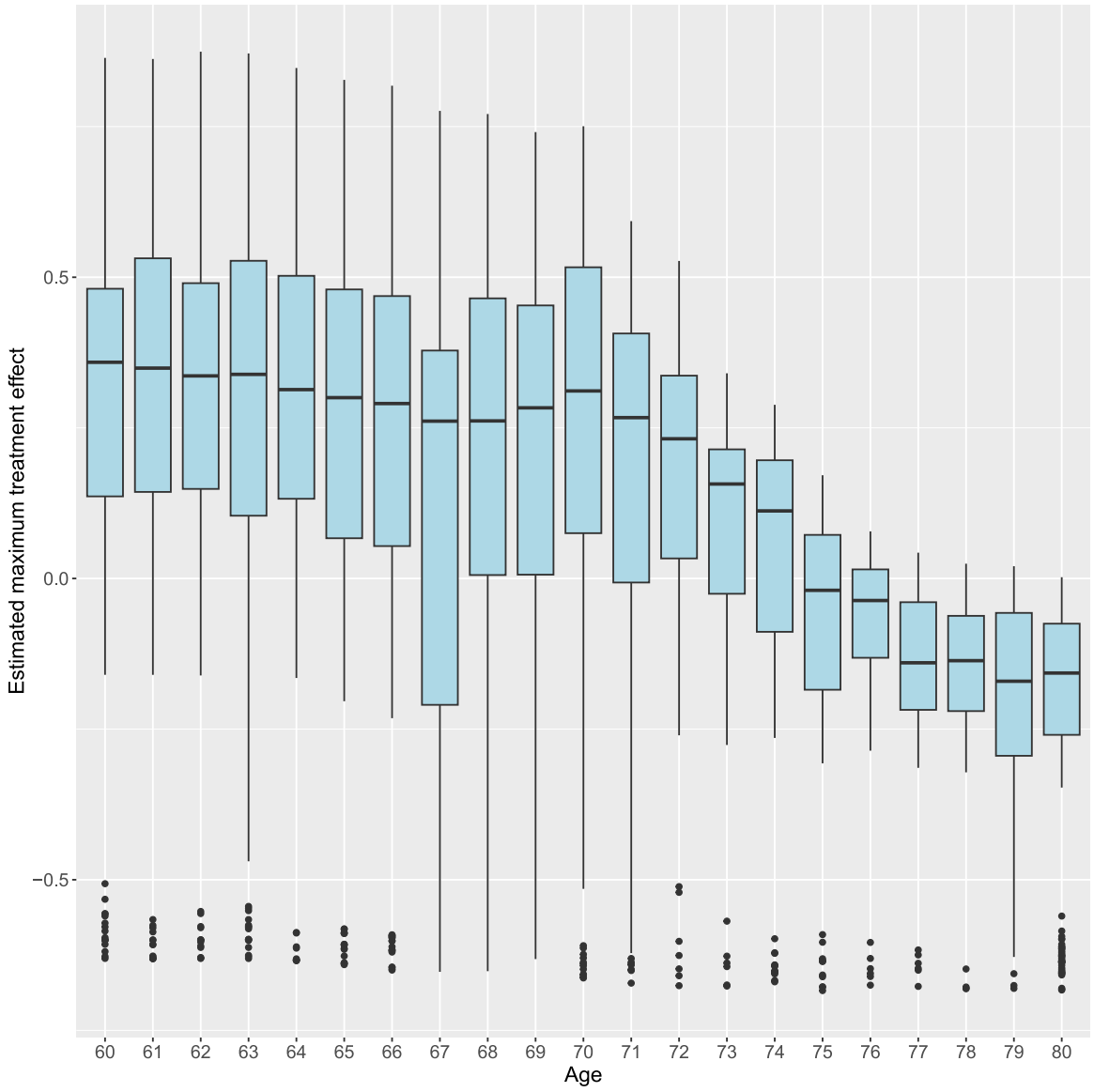

Fig. 13. Distribution of estimated maximum treatment effects at each age.

```
  ylab="Estimated maximum effect",main="
    HETMaxQuad Method (test set all In-Bag
    estimations)")
abline(a=0,b=1)

plot(datrain$tau3,predtrain,xlab="True maximum
    effect",
  ylab="Estimated maximum effect",main="
    HETMaxQuad Method (training set OOB
    estimations)")
abline(a=0,b=1)

# Computing time comparison
# By default, all cores are used for computation
start_time <- Sys.time()
mod = HETMaxQuad(f(y, g) ~ x1+x2+x3+x4+x5, data
    = datrain ,ntree = 100,
  samptype ="swr",nodesize=15,mtry=5)
end_time <- Sys.time()
end_time - start_time

# We can select the number of cores to use. Here
     we use 1.
start_time <- Sys.time()
mod1=HETMaxQuad(f(y, g) ~ x1+x2+x3+x4+x5, data =
     datrain ,ntree = 100,
  samptype ="swr",nodesize=15,mtry=5,iterf.
    nbCore=1)
end_time <- Sys.time()
end_time - start_time
```

B.2. Estimation of the slope of the treatment curve for a continuous response and a continuous treatment with the functions HET and CMB

In this example, we estimate the treatment effect $\tau_2(x)$ defined in Section 2.5. We use local centering of the treatment and response. The function `generatedatatau2` generates the data according to DGP linear described in Section 3.2. However, note that we are not estimating the maximum effect but the slope of the assumed linear treatment curve.

```
generatedatatau2=function(n)
{
# INPUT:
# n =sample size
```

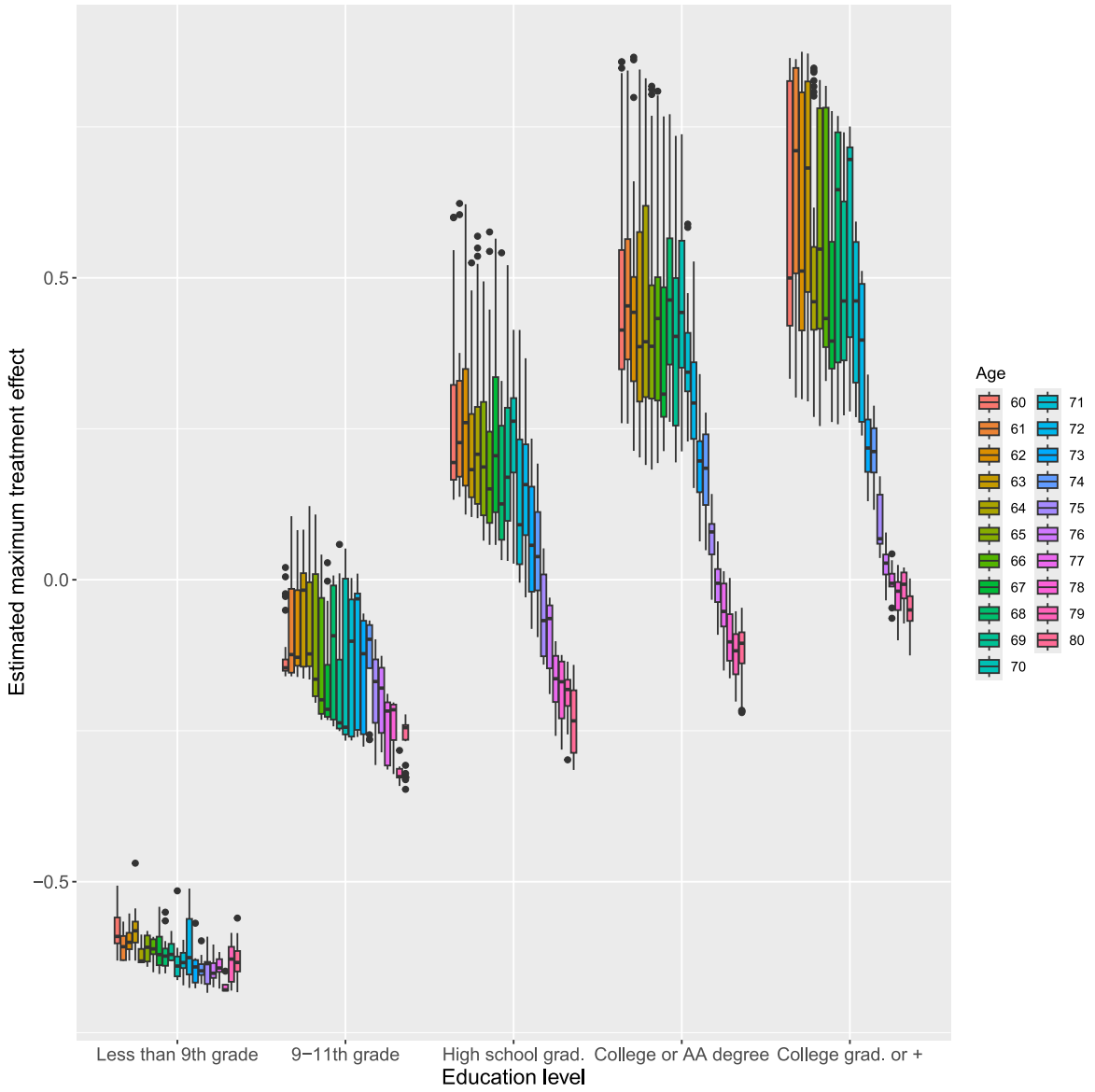

Fig. 14. Distribution of estimated maximum treatment effects at each age and education level combination.

```
# OUTPUT:
# y = response
# g = treatment value (continuous)
# x1,...,x5 = covariates
# tau2 = treatment effect (slope)

x1 = runif(n)*10
x2 = runif(n)*10
x3 = runif(n)*10
x4 = runif(n)*10
x5 = runif(n)*10
g = runif(n)
b0=(x1<=5)*(x3<=5)*1 + (x1<=5)*(x3>5)*2 + (x1>5)
    *(x2<=5)*3 + (x1>5)*(x2>5)*4
b1=((x4-5)^2)/5
y=b0 + g*b1  + rnorm(n,0,.5)
tau2=b1
dat = data.frame(y,g,x1,x2,x3,x4,x5,tau2)
dat
}

library(ITERF)
```

```
# Training and test sample sizes
ntrain=3000
ntest=200

# Data generation
set.seed(345)
datrain=generatedatatau2(ntrain)
datest=generatedatatau2(ntest)

par(mfrow=c(1,2))

# Fit the random forest with the HET method
modhet = HET(f(y, g) ~ x1+x2+x3+x4+x5, data =
    datrain ,ntree = 100,
  samptype ="swr",nodesize=15,mtry=5,
  iterf.CenterResponse = TRUE, iterf.
    CenterTreatment = TRUE)
# Compute OOB estimations for test data
predhet = predict(modhet, newdata=datest,OOB =
    TRUE, obsUniqueInTree = TRUE)$predicted

# Fit the random forest with the CMB method
```

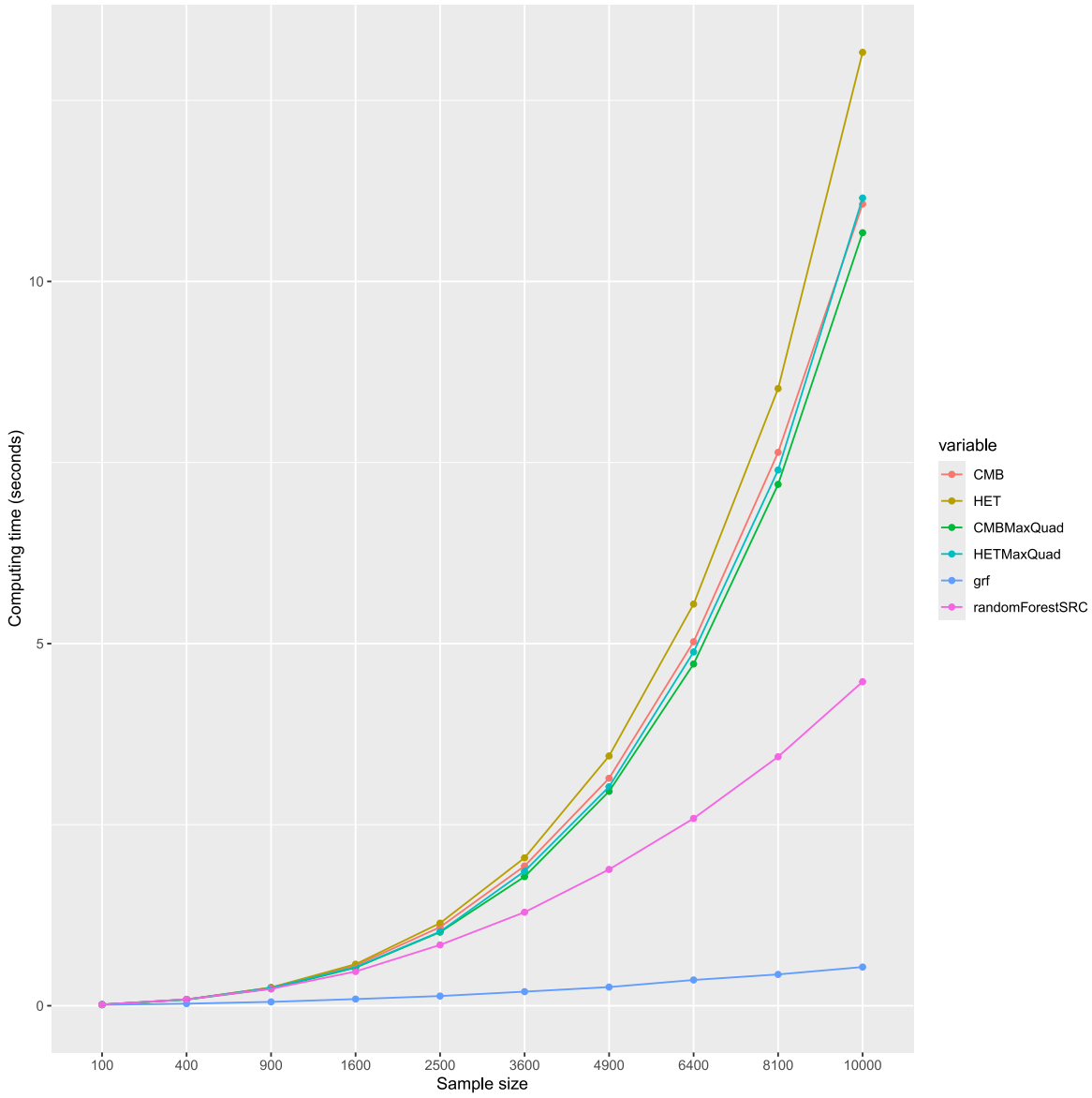

Fig. 15. Computing time comparisons using the default multi-cores.

```
modcmb = CMB(f(y, g) ~ x1+x2+x3+x4+x5, data =
    datrain ,ntree = 100,
  samptype ="swr",nodesize=15,mtry=5,
  iterf.CenterResponse = TRUE, iterf.
    CenterTreatment = TRUE)
# Compute OOB estimations for test data
predcmb = predict(modcmb, newdata=datest,OOB =
    TRUE, obsUniqueInTree = TRUE)$predicted

# Plot true versus estimated values
par(mfrow=c(1,2))

plot(datest$tau2,predhet,xlab="True treatment
    effect (slope)",
  ylab="Estimated treatment effect (slope)",main
    ="HET Method")
abline(a=0,b=1)

plot(datest$tau2,predcmb,xlab="True treatment
    effect (slope)",
  ylab="Estimated treatment effect (slope)",main
    ="CMB Method")
abline(a=0,b=1)
```

B.3. Estimation of the CATE for survival data and a binary treatment with the function HETSurv

In this example, we estimate the treatment effect $\tau_1(x)$ defined in Section 2.4. The function `generatedatatau1` generates the data.

```
generatedatatau1=function(nc,nt)
{
# INPUT:
# nc= control sample size (binary treatment)
# nt= treatment sample size (binary treatment)
# OUTPUT:
# t = observed event time
# c = censoring indicator (1=event)
# g = binary treatment indicator
# x1,...,x10 = covariates
# tau1 = CATE

library(MASS)
```

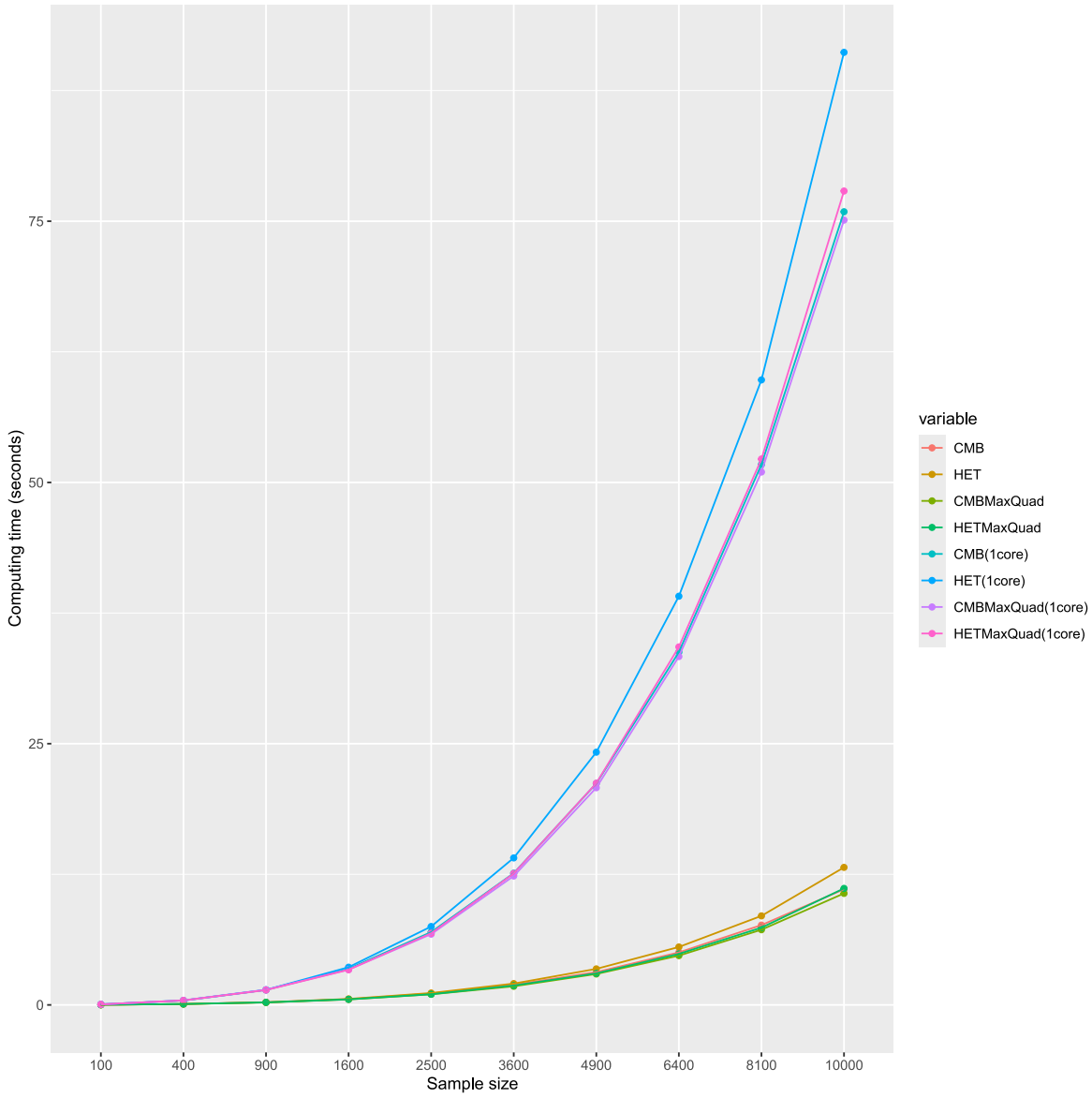

Fig. 16. Computing time comparisons between multi-cores and a single core.

```
dat=data.frame(mvrnorm(nc+nt, rep(0, 10), diag
    (10)))
names(dat)=paste("x",1:10,sep="")
co=3+(dat$x1+dat$x2+dat$x3+dat$x4+dat$x2^2+dat$
    x2*dat$x4)/10
dat$tau1=(dat$x7+2*dat$x5+4*dat$x1^2+abs(dat$x2)
    +dat$x1*dat$x2+abs(dat$x3^3)+exp(dat$x8))/10
dat$y=0
dat$y[1:nc]=co[1:nc]+rexp(nc,2)
dat$y[(nc+1):(nc+nt)]=co[(nc+1):(nc+nt)]+dat$
    tau1[(nc+1):(nc+nt)]+rexp(nt,2)
dat$g=c(rep(0,nc),rep(1,nt))
dat$v=rexp(nc+nt,.05)
dat$t=dat$y*(dat$y<=dat$v)+dat$v*(dat$y>dat$v)
dat$c=1-as.numeric(dat$y>dat$v)
dat$y=NULL
dat$v=NULL
dat
}

# Generate data
set.seed(345)
datrain=generatedatatau1(nc=2000,nt=2000)
datest=generatedatatau1(nc=100,nt=100)

summary(datrain)

#Proportion of censoring in the training data
mean(1-datrain$c)

par(mfrow=c(1,2))

# Histogram of CATE for the training data
hist(datrain$tau1,main="CATE")

library(ITERF)

# build forest
obj <- HETSurv(f(t, c, g) ~ x1+x2+x3+x4+x5+x6+x7
    +x8+x9+x10, mtry=10,nodesize=20,
   data = datrain ,ntree = 100,samptype ="swr")

# Compute OOB estimations for test data
```

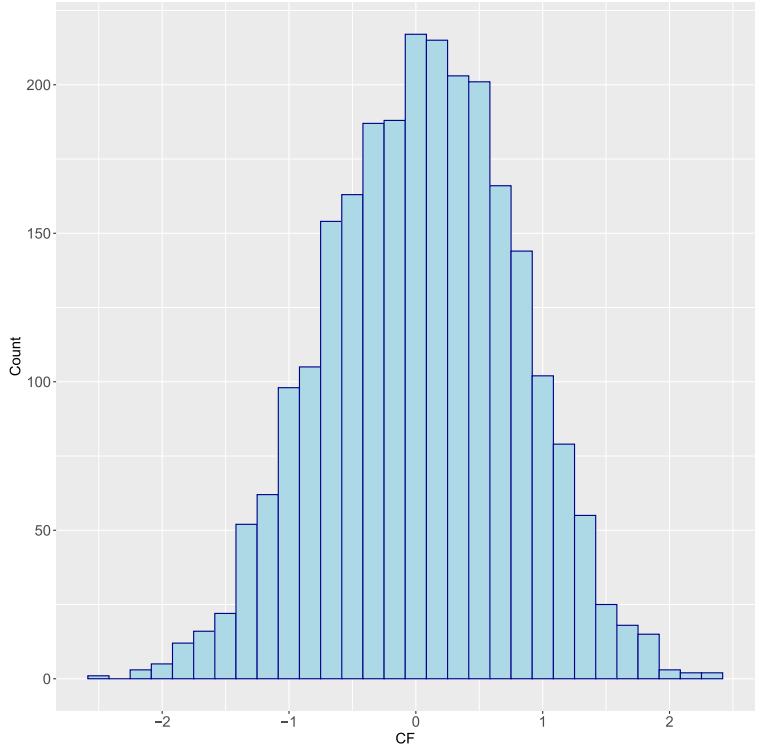

Fig. 17. Histogram of CF (Cognitive function), the response variable.

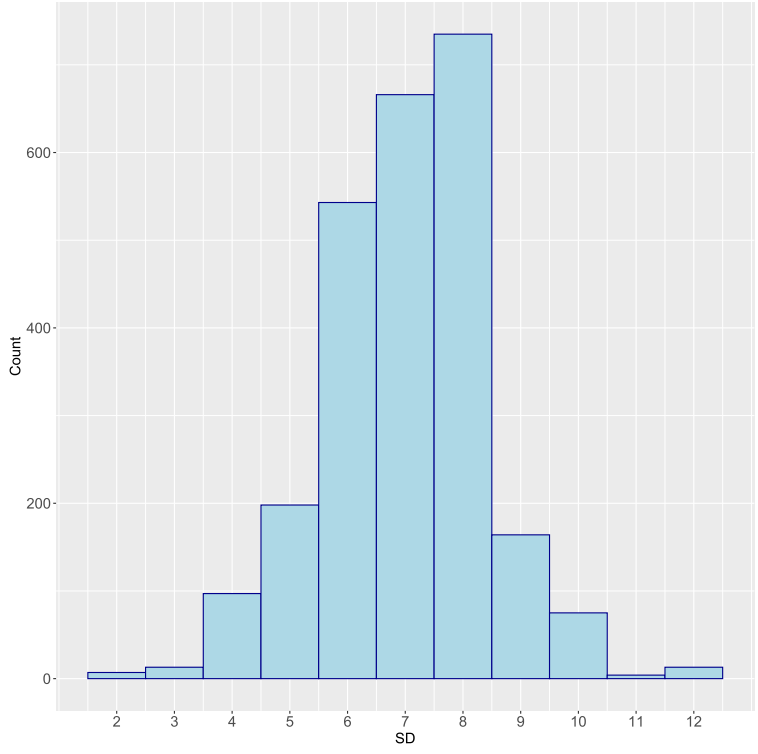

Fig. 18. Histogram of SD, the treatment variable.

```
predoob= predict(obj, newdata=datest,OOB = TRUE,
     obsUniqueInTree = TRUE)

# Plot true versus estimated values
plot(datest$tau1,predoob$predicted,xlab="True
    treatment effect",ylab="Estimated treatment
    effect",main="HETSurv method")
abline(a=0,b=1)
```

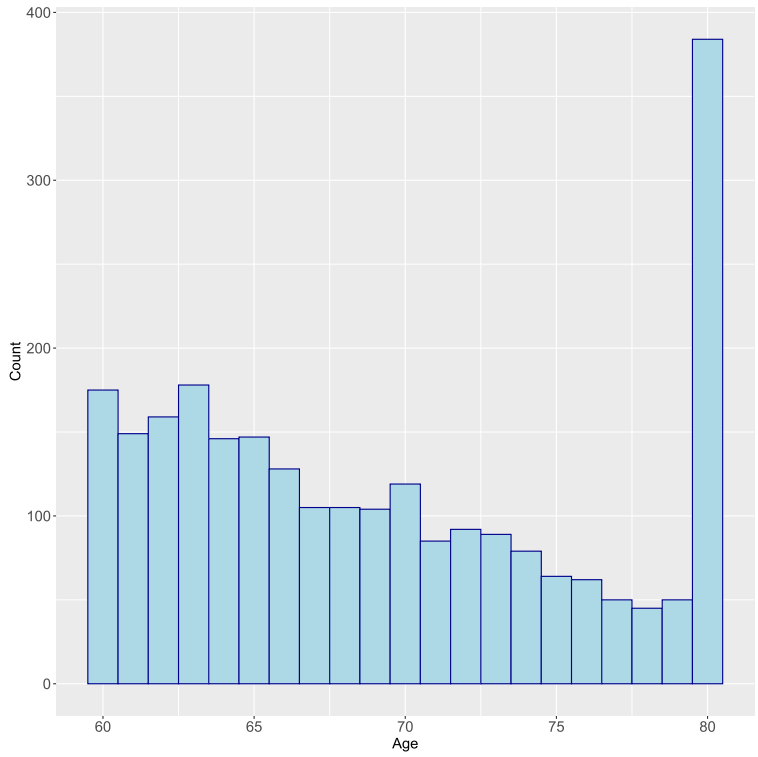

Fig. 19. Histogram of Age.

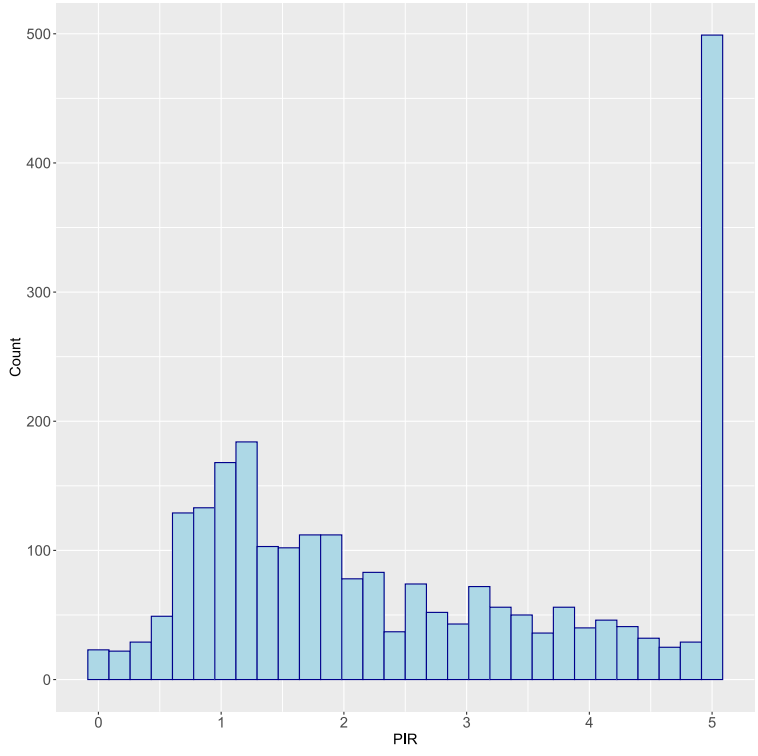

Fig. 20. Histogram of Poverty Income Ratio (PIR).

Appendix C. Extended Kaplan–Meier estimator

For a given set of data, let $\hat{S}_{KM}(t)$ be KM estimate and let t_{max} be the largest value where it is defined. The split rule defined in Section 2.4 requires the integration of $\hat{S}_{KM}(t)$ to obtain the estimated mean survival time. If $\hat{S}_{KM}(t_{max}) = 0$, then this is trivial. However, in some cases, the KM estimator is undefined past a certain value, that is $\hat{S}_{KM}(t_{max}) > 0$. To circumvent that, we add a tail from the exponential distribution at the end of the KM estimator.

When $\hat{S}_{KM}(t_{max}) > 0$, let $\hat{\lambda}$ be the value such that $\hat{S}_{KM}(t_{max}) = S_{\hat{\lambda}}(t_{max})$, where $S_{\lambda}(t) = \exp(-\lambda t)$ is the survival function from the

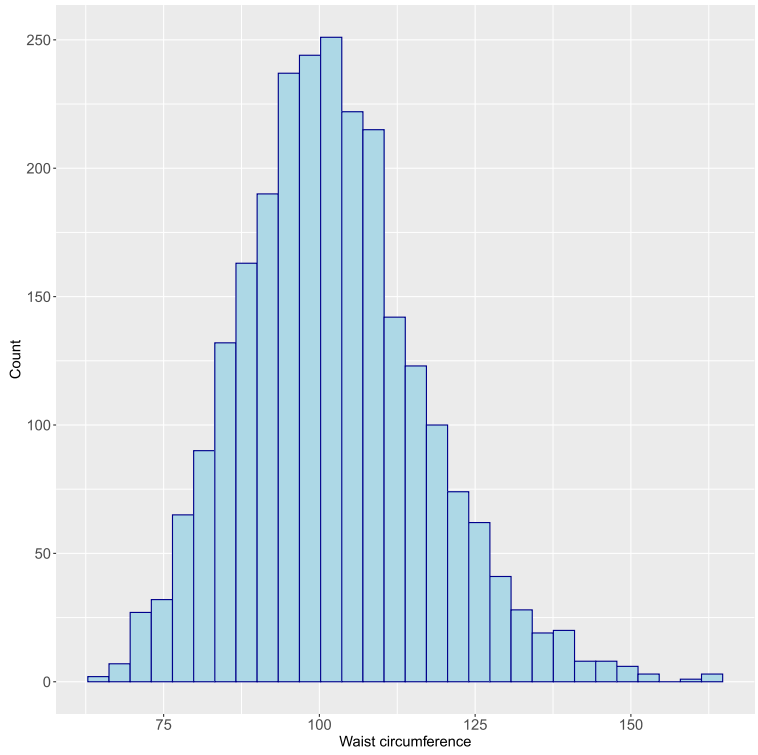

Fig. 21. Histogram of Waist Circumference.

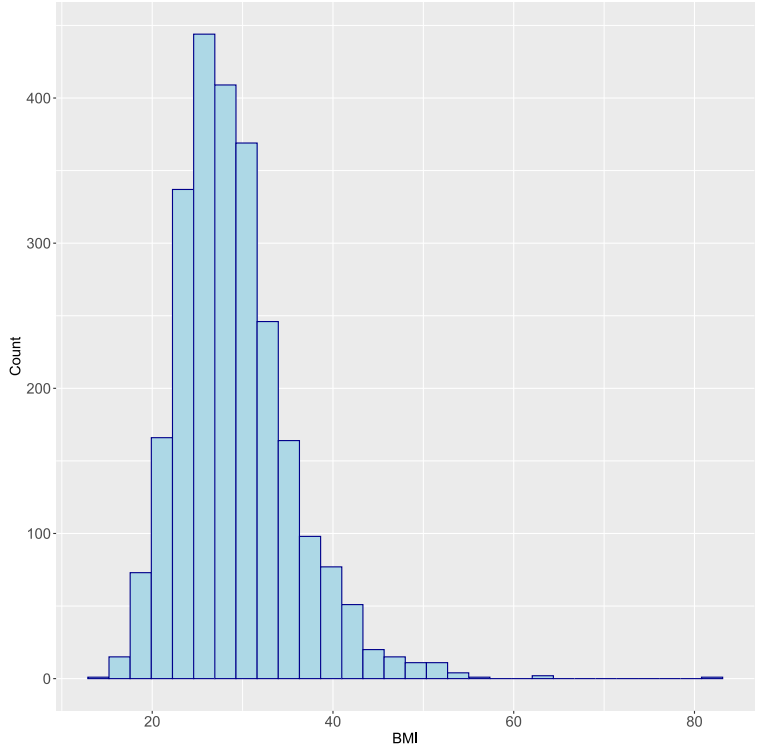

Fig. 22. Histogram of BMI.

exponential distribution with parameter λ. The KM estimator is extended beyond t_{max} by $S_{\hat{\lambda}}(t)$. That is, we define $\hat{S}_{KM}(t) = S_{\hat{\lambda}}(t)$ for all $t > t_{max}$, and we call the resulting function the extended KM estimator. Integrating this extended KM estimator gives the estimated mean survival time. In practice, this amounts to integrating the original KM up to t_{max}, and add $\int_{t_{max}}^{\infty} S_{\hat{\lambda}}(t)dt = \exp(\hat{\lambda}t_{max})/\hat{\lambda}$.

Appendix D. Computing time comparisons

Parallel multi-core execution is enabled by default in `ITERF`. In this experiment, we use the DGP linear setting (DGP 2) without additional noise covariates, as described in Section 3.2. We consider the same ten training sample sizes used in the simulation study, specifically $n = k^2$

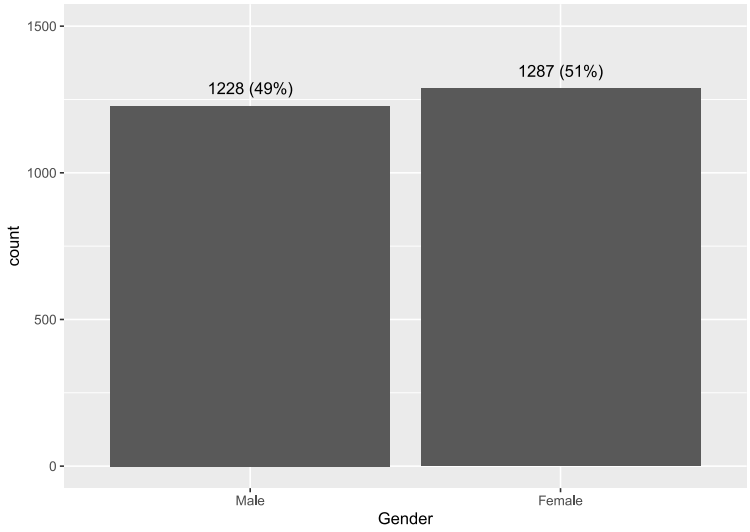

Fig. 23. Distribution of Gender.

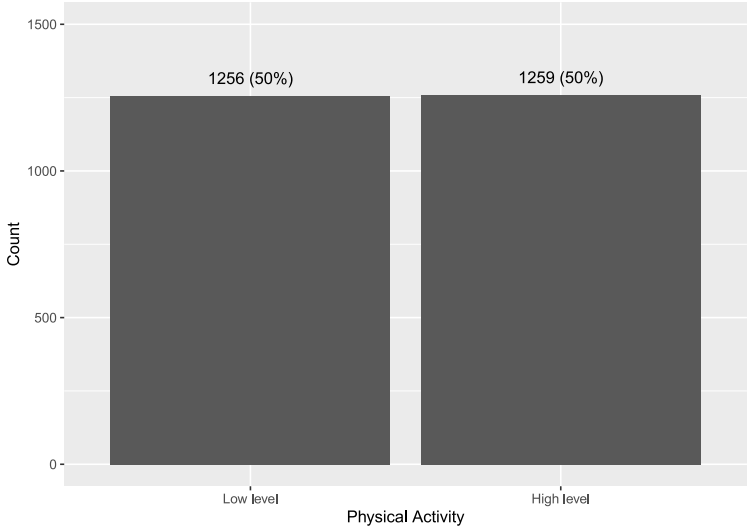

Fig. 24. Distribution of Physical Activity.

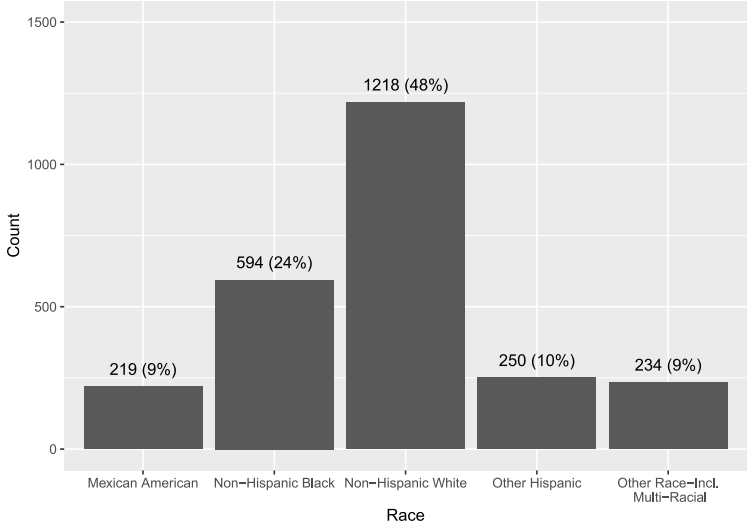

Fig. 25. Distribution of Race.

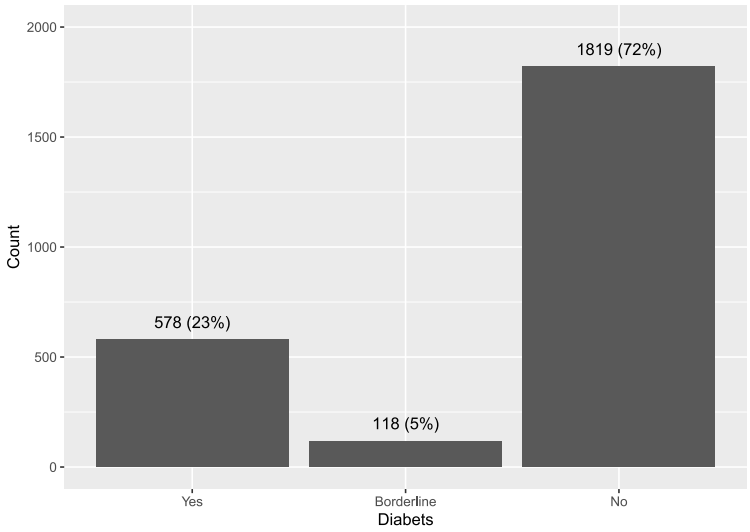

Fig. 26. Distribution of Diabetes.

for $k = 10, 20, \ldots, 100$. For each sample size, we perform 10 simulation runs and compute the average time required to build a random forest with 100 trees across different methods. The first four methods evaluated are implemented in `ITERF`: `CMB`, `HET`, `CMBMaxQuad`, and `HETMaxQuad`. The first two estimate the slope of the treatment effect, while the latter two estimate the maximum treatment effect. In each node, `CMB` and `HET` fit a linear model using the treatment as the covariate, whereas `CMBMaxQuad` and `HETMaxQuad` fit a quadratic model. We also include the function `causal_forest` from the `grf` package, which estimates the slope of the treatment effect. As mentioned in Section 2.8, `HET` is conceptually similar to `causal_forest`. However, `causal_forest` uses an approximate split rule to improve efficiency. As a baseline comparison, we include a standard random forest built using the `rfsrc` function from the `randomForestSRC` package, which employs a basic least-squares split rule. All experiments are conducted on a desktop equipped with an Intel Xeon W-2133 processor (3.60 GHz) and 64 GB of RAM.

Fig. 15 illustrates the average computing time for forest construction as a function of sample size. For instance, with $n = 10{,}000$, the `HET` method takes approximately 13 s to build a forest. The most direct comparison is with `randomForestSRC`, since `ITERF` shares the same underlying architecture. The key difference lies in the split rule computation: `ITERF` requires additional calculations to fit linear (`CMB`, `HET`) or quadratic (`CMBMaxQuad`, `HETMaxQuad`) models within each node, whereas `randomForestSRC` uses a simpler least-squares criterion. These extra computations account for the observed differences in runtime between `ITERF` and `randomForestSRC`. Notably, `grf` emerges as the fastest method in this experiment.

The support for parallel processing in `ITERF` is a significant advantage. Fig. 16 compares the average computing time when using all available cores versus restricting execution to a single core via the option `iterf.nbCore=1`. For example, with $n = 10{,}000$, the `HET` method takes about 91 s using a single core, compared to just 13 s with multi-core execution.

Appendix E. Additional details about the real data example

In this appendix, we provide more details about the real data example presented in Section 4.

E.1. Data extracted from NHANES

In the example, we use data from the NHANES 2011–2014 files. Multiple datasets are combined using the variable `SEQN`, which represents the respondent sequence number and is available in all datasets.

Table 2
Files used from the NHANES database.

Filename	File description	Year
DEMO_G	Demographic Variables & Sample Weights	2011–2012
DEMO_H	Demographic Variables & Sample Weights	2013–2014
BMX_G	Body Measures	2011–2012
BMX_H	Body Measures	2013–2014
PAQ_G	Physical Activity	2011–2012
PAQ_H	Physical Activity	2013–2014
DIQ_G	Diabetes	2011–2012
DIQ_H	Diabetes	2013–2014
SLQ_G	Sleep Disorders	2011–2012
SLQ_H	Sleep Disorders	2013–2014
CFQ_G	Cognitive Functioning	2011–2012
CFQ_H	Cognitive Functioning	2013–2014

The list of files is shown in Table 2, while Table 3 provides the variables extracted from each dataset.

E.2. Details about the response variable CF and the variable physical activity

Following [18], the cognitive function (`CF`) score is computed as the average of four normalized test scores, each assessing a distinct cognitive domain: short-term memory and learning (`IR`), memory retention (`DR`), verbal fluency and semantic memory (`AF`), and executive functioning and processing speed (`DSST`).

In detail, `IR` is calculated as the average of three recall tests:

$$\texttt{IR} = (\texttt{CFDCST1} + \texttt{CFDCST2} + \texttt{CFDCST3})/3,$$

where `CFDCST1`, `CFDCST2` and `CFDCST3` represent the recall test results from the NHANES Cognitive Functioning Questionnaire. Each score is normalized prior to computing their average. `DR` corresponds to the normalized Delayed Recall score (CFDCSR), `AF` to the normalized Animal Fluency test score (CFDAST), and `DSST` to the normalized Digit Symbol test score (CFDDPP).

The cognitive function score is then computed as:

$$\texttt{CF} = (\texttt{IR} + \texttt{DR} + \texttt{AF} + \texttt{DSST})/4.$$

Following [18], physical activity is quantified using MET-min/week (Metabolic Equivalent of Task). Individuals with a MET value greater than or equal to 600 are classified as having a high level of physical activity; otherwise, they are considered to have a low level of physical activity. To calculate the MET, we adopt the method described in [31]:

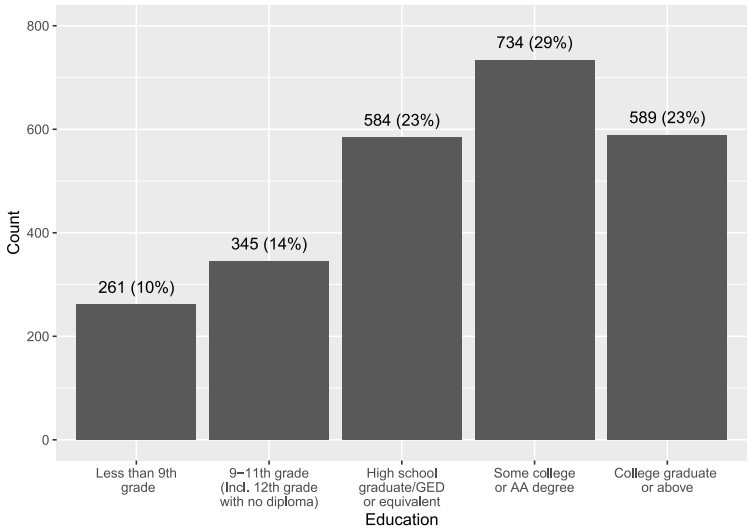

Fig. 27. Distribution of Education.

Table 3
Variables extracted from NHANES files.

File name	Variable name	Variable description
DEMO	RIAGENDR	Gender of the participant
DEMO	RIDAGEYR	Age in years of the participant at the time of screening. (Individuals 80 and over are topcoded at 80 years of age.)
DEMO	RIDRETH1	Race/Hispanic origin
DEMO	DMDEDUC2	Education level - Adults 20+
DEMO	INDFMPIR	Ratio of family income to poverty (topcoded at 5)
BMX	BMXBMI	Body Mass Index (kg/m**2)
BMX	BMXWAIST	Waist Circumference (cm)
PAQ	PAD615	Minutes vigorous-intensity work
PAQ	PAD630	Minutes moderate-intensity work
PAQ	PAD645	Minutes walk/bicycle for transportation
PAQ	PAD660	Minutes vigorous recreational activities
PAQ	PAD675	Minutes moderate recreational activities
PAQ	PAQ610	Days vigorous work
PAQ	PAQ625	Number of days moderate work
PAQ	PAQ640	Number of days walk or bicycle
PAQ	PAQ655	Days vigorous recreational activities
PAQ	PAQ670	Days moderate recreational activities
DIQ	DIQ010	Doctor told you have diabetes
SLQ	SLD010H	How much sleep do you get (hours)?
CFQ	CFDCST1	CERAD: Score Trial 1 Recall
CFQ	CFDCST2	CERAD: Score Trial 2 Recall
CFQ	CFDCST3	CERAD: Score Trial 3 Recall
CFQ	CFDCSR	CERAD: Score Delayed Recall
CFQ	CFDAST	Animal Fluency: Score Total
CFQ	CFDDS	Digit Symbol Coding: Sample Practice Pretest

MET = number of minutes of activity per day per activity type × number of days with this activity type in a week × weight of activity type. Activity types are weighted as follows:

- Active work-related activity (high-intensity): 8.0
- Moderate work-related activity (moderate-intensity): 4.0
- Walking or biking for transportation: 4.0
- Vigorous recreational physical activity: 8.0
- Moderate leisure-time physical activity: 4.0.

E.3. Descriptive statistics for the sample used in the analysis

As described in Section 4, the sample used for the analysis consists of 2515 observations. All subsequent statistics and figures are based on this sample. Table 4 reports descriptive statistics for the continuous variables, while the distributions of all variables are illustrated using histograms and bar plots in Figs. 17 to 27. In the NHANES database, the variable age is topcoded at 80 and the variable PIR is topcoded at 5.

E.4. Other figure

Fig. 28 shows that the estimated treatment effects obtained from the method `CMBMaxQuad` are very similar to the ones obtained from the method `HETMaxQuad`. Hence, only the ones from `HETMaxQuad` are presented and discussed.

Table 4
Descriptive statistics for the continuous variables (n = 2515 observations from NHANES).

Variable	minimum	maximum	mean	median	standard deviation
CF (response variable)	−2.43	2.41	0.04	0.06	0.75
SD (treatment variable)	2	12	7.02	7	1.42
Covariates					
age	60	80	69.28	68	6.73
poverty income ratio (PIR)	0	5	2.62	2.19	1.6
waist circumference	65.2	163.7	102.07	101.1	14.75
body mass index (BMI)	14.2	82.1	29.05	28	6.24

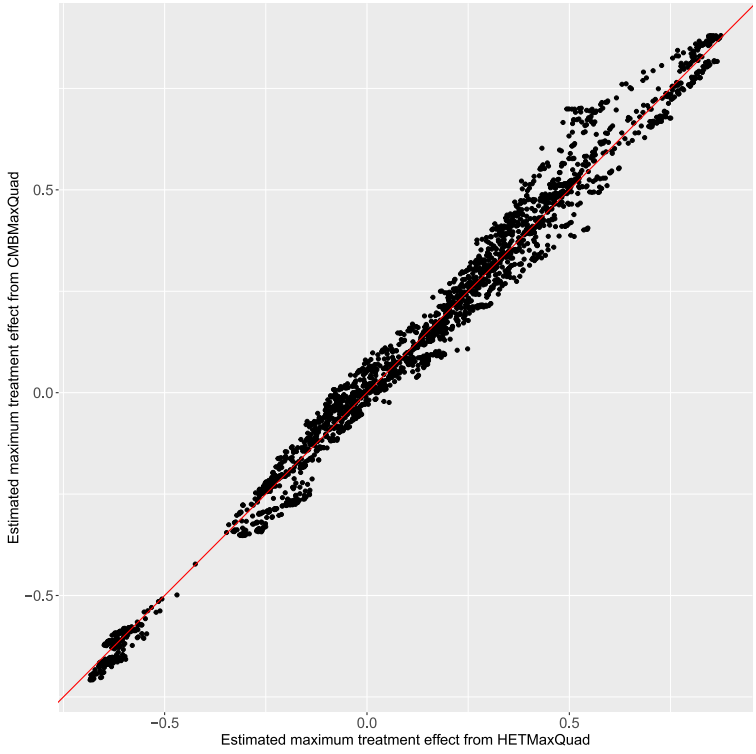

Fig. 28. Comparison of the estimated maximum treatment effects obtained from the models `HETMaxQuad` and `CMBMaxQuad`.

References

[1] A. Hu, Heterogeneous treatment effects analysis for social scientists: A review, Soc. Sci. Res. 109 (2023) 102810.
[2] W. Zhang, J. Li, L. Liu, A unified survey of treatment effect heterogeneity modelling and uplift modelling, ACM Comput. Surv. 54 (8) (2021) 1–36.
[3] G.W. Imbens, D.B. Rubin, Causal Inference in Statistics, Social, and Biomedical Sciences, Cambridge University Press, 2015.
[4] A. Curth, R.W. Peck, E. McKinney, J. Weatherall, M. van Der Schaar, Using machine learning to individualize treatment effect estimation: Challenges and opportunities, Clin. Pharmacol. Ther. 115 (4) (2024) 710–719.
[5] J. Abécassis, É. Dumas, J. Alberge, G. Varoquaux, From prediction to prescription: Machine learning and causal inference for the heterogeneous treatment effect, Annu. Rev. Biomed. Data Sci. 8 (2025).
[6] L. Breiman, Random forests, Mach. Learn. 45 (1) (2001) 5–32.
[7] S.R. Künzel, J.S. Sekhon, P.J. Bickel, B. Yu, Metalearners for estimating heterogeneous treatment effects using machine learning, Proc. Natl. Acad. Sci. 116 (10) (2019) 4156–4165.
[8] S. Wager, S. Athey, Estimation and inference of heterogeneous treatment effects using random forests, J. Amer. Statist. Assoc. 113 (523) (2018) 1228–1242.
[9] S. Athey, J. Tibshirani, S. Wager, Generalized random forests, Ann. Statist. 47 (2) (2019) 1148–1178.
[10] J. Tibshirani, S. Athey, E. Sverdrup, S. Wager, Grf: Generalized random forests, 2024, R package version 2.4.
[11] P. Rehill, How do applied researchers use the causal forest? A methodological review, Int. Stat. Rev. (2025).
[12] S. Tabib, D. Larocque, Non-parametric individual treatment effect estimation for survival data with random forests, Bioinformatics 36 (2) (2020) 629–636.
[13] S. Tabib, D. Larocque, Comparison of random forest methods for conditional average treatment effect estimation with a continuous treatment, Stat. Methods Med. Res. 33 (11–12) (2024) 1952–1966.
[14] H. Ishwaran, U.B. Kogalur, Fast unified random forests for survival, regression, and classification (RF-SRC), 2025, R package version 3.4.1.
[15] H. Ishwaran, U. Kogalur, E. Blackstone, M. Lauer, Random survival forests, Ann. Appl. Stat. 2 (3) (2008) 841–860.
[16] H. Ishwaran, U. Kogalur, Random survival forests for r, R News 7 (2) (2007) 25–31.
[17] OpenMP Architecture Review Board, OpenMP application program interface version 3.0, 2008.
[18] P. Qiu, C. Dong, A. Li, J. Xie, J. Wu, Exploring the relationship of sleep duration on cognitive function among the elderly: a combined NHANES 2011–2014 and mendelian randomization analysis, BMC Geriatr. 24 (1) (2024) 935.
[19] Centers for disease control and prevention (CDC), national center for health statistics (NCHS). National health and nutrition examination survey data, 2011–2014, https://wwwn.cdc.gov/nchs/nhanes/. Hyattsville, MD: U.S. Department of Health and Human Services, Centers for Disease Control and Prevention.
[20] L. Ale, R. Gentleman, T.F. Sonmez, D. Sarkar, C. Endres, Nhanesa: achieving transparency and reproducibility in NHANES research, Database Apr 15 (2024) http://dx.doi.org/10.1093/database/baae028.
[21] S. Wood, Fast stable restricted maximum likelihood and marginal likelihood estimation of semiparametric generalized linear models, J. R. Stat. Soc. (B) 73 (1) (2011) 3–36, http://dx.doi.org/10.1111/j.1467-9868.2010.00749.x.
[22] C. Alakuş, D. Larocque, S. Jacquemont, F. Barlaam, C.-O. Martin, K. Agbogba, S. Lippé, A. Labbe, Conditional canonical correlation estimation based on covariates with random forests, Bioinformatics 37 (17) (2021) 2714–2721.
[23] C. Spanbauer, R. Sparapani, Nonparametric machine learning for precision medicine with longitudinal clinical trials and Bayesian additive regression trees with mixed models, Stat. Med. 40 (11) (2021) 2665–2691.
[24] A.D. Meid, A. Gerharz, A. Groll, Machine learning for tumor growth inhibition: Interpretable predictive models for transparency and reproducibility, CPT: Pharmacomet. Syst. Pharmacol. 11 (3) (2022) 257.

[25] F.J. Bargagli-Stoffi, K. De Witte, G. Gnecco, Heterogeneous causal effects with imperfect compliance: A Bayesian machine learning approach, Ann. Appl. Stat. 16 (3) (2022) 1986–2009.
[26] J. Sexton, P. Laake, Standard errors for bagged and random forest estimators, Comput. Statist. Data Anal. 53 (3) (2009) 801–811.
[27] S. Wager, T. Hastie, B. Efron, Confidence intervals for random forests: The jackknife and the infinitesimal jackknife, J. Mach. Learn. Res. 15 (1) (2014) 1625–1651.
[28] H. Ishwaran, M. Lu, Standard errors and confidence intervals for variable importance in random forest regression, classification, and survival, Stat. Med. 38 (4) (2019) 558–582.
[29] M.L. Petersen, K.E. Porter, S. Gruber, Y. Wang, M.J. Van Der Laan, Diagnosing and responding to violations in the positivity assumption, Stat. Methods Med. Res. 21 (1) (2012) 31–54.
[30] E.E. Moodie, J. Schulz, A simple diagnostic for the positivity assumption for continuous exposures, Stat. Med. 44 (15–17) (2025) e70194.
[31] L. Tian, C. Lu, W. Teng, Association between physical activity and thyroid function in American adults: a survey from the NHANES database, BMC Public Health 24 (1) (2024) 1277.